

Exploiting Physical Dynamics to Detect Actuator and Sensor Attacks in Mobile Robots

Pinyao Guo, Hunmin Kim, Nurali Virani, Jun Xu, Minghui Zhu and Peng Liu
 Pennsylvania State University
 University Park, PA, 16802

ABSTRACT

Mobile robots are cyber-physical systems where the cyberspace and the physical world are strongly coupled. Attacks against mobile robots can transcend cyber defenses and escalate into disastrous consequences in the physical world. **In this paper, we focus on the detection of active attacks that are capable of directly influencing robot mission operation.** Through leveraging physical dynamics of mobile robots, we develop **RIDS**, a novel robot intrusion detection system that can detect actuator attacks as well as sensor attacks for nonlinear mobile robots subject to stochastic noises. **We implement and evaluate a RIDS on Khepera mobile robot against concrete attack scenarios via various attack channels including signal interference, sensor spoofing, logic bomb, and physical damage.** Evaluation of 20 experiments shows that the averages of false positive rates and false negative rates are both below 1%. Average detection delay for each attack remains within 0.40s.

1 INTRODUCTION

Recent years have witnessed a rapid growth in the robotics industry. According to International Data Corporation [24], global spending on robotics and related services will reach \$135 billion in 2019. The sheer size of robotics volume is mainly accounted from defense and security, agricultural, medical care, and manufacturing applications [1]. Recent market predicts a major growth in household and entertainment applications [5]. Mobile robots, as a typical type of robot systems, have capabilities of movement in particular work environments and carry out specific missions. Some representative mobile robots include household cleaning robots such as Roomba, military surveillance drones such as Global Hawk, aerial photography unmanned aerial vehicles (UAV) such as DJI Phantom, Amazon warehouse robots, etc. Major tech companies, e.g., Google, Uber, and Tesla are leading intensive development of autonomous cars to replace human drivers in near future [38].

The popularity of this emerging technology introduces new security threats to the community. Unlike traditional cyber systems such as computers or mobile phones, mobile robots are characterized by a strong coupling of the cyberspace and the physical world in which they operate. Mobile robots equip sensors, actuators, and electronic control units (ECUs). In a typical control iteration, sensors (e.g., GPS, accelerometer) measure the states (e.g., position, orientation, velocity, etc.) of robots and their surrounding world, and feed the readings to ECUs. ECUs generate control commands based on mission specifications, and actuators (e.g., rotor, wheel) execute them in the physical world. Mobile robots inherit vulnerabilities from their cyber components, and such vulnerabilities can be exploited by adversaries to transcend cyber defenses and further escalate into disastrous consequences in the physical world. Recently,

researchers demonstrated several remote hacks into a Jeep Cherokee [40] and Tesla newest models [2], and were able to control their actuators such as steering wheels and gas pedals. Moreover, actuators and sensors introduce extra attack surfaces and vulnerabilities into mobile robots. In 2011, an American surveillance drone was brought down by Iranian cyber warfare unit through GPS spoofing attacks [61]. In 2013, a multi-million yacht was demonstrated to be hijacked and controlled using spoofed GPS signals [67]. Many missions of robots are safety critical. Hence, it becomes an urgent issue to ensure the security of mobile robots.

In this paper, we focus on intrusion detection for mobile robots. We consider attacks that are capable of transcending cyber defenses, actively altering robot behavior and causing damages in the physical world. Down to attack consequences, active attacks can be classified into actuator attacks and sensor attacks. *Actuator attacks*, e.g., steering wheel take-over, directly alter control commands executed by robot actuators. *Sensor attacks*, e.g., GPS spoofing, alter authentic sensor readings received by controllers.

Cyber-layer or cyberspace intrusion detection has been studied extensively in past decades. Traditional host-based [60, 63] and network-based [47, 53] IDSs monitor cyberspace behaviors, e.g., system calls, network events, etc. However, attacks launched through physical channels cannot be detected, since no abnormal cyberspace behavior would be triggered and captured. Wireless sensor network intrusion detection approaches [34, 37] leverage sensor redundancy to do majority voting on sensor readings and detect inconsistencies between each other. These approaches resort to particular Byzantine thresholds [8] on the number of uncorrupted sensors. When a powerful attack is launched that compromises more sensors than the threshold, the detection fails.

Mobile robots are cyber-physical systems. Beyond the knowledge audited by cyber-layer intrusion detection approaches, they can also access to a *second* source of knowledge learned from interacting with the physical world. In particular, the physical dynamics of mobile robots impose constraints on the maneuver of mobile robots. These constraints can be leveraged as a detection vector to provide essential information that reflects ground truth statuses. The second source of knowledge is neither obtained nor used in cyber-layer intrusion detection approaches. Noticeably, the information provided by physical dynamics allows for detecting sensor and actuator attacks without resorting to majority voting or Byzantine thresholds.

In this paper, we propose a robot intrusion detection system (RIDS) for nonlinear mobile robots subject to stochastic noises, which leverages the physical dynamics of mobile robots. The proposed RIDS does not assume any sensor or actuator is clean. It is able to detect, pinpoint and quantify sensor and actuator attacks when not all sensors are simultaneously corrupted. The detection

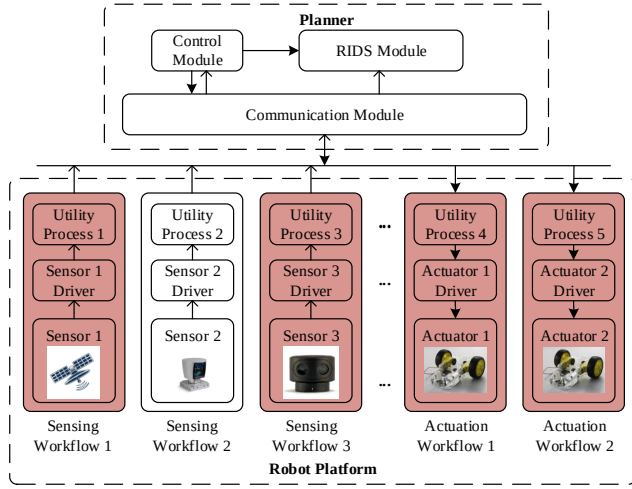


Figure 1: Mobile robot system model which consists of a robot platform and a planner. The physical world is perceived by different sensors. Sensor readings go through different processing in utility processes and reach to the planner through a communication module. Control commands are generated from the planner and executed via actuators. (Hollow arrows stand for sensor reading data, and filled arrows stand for control command data.)

capabilities are produced by explicitly leveraging physical dynamics of mobile robots.

Our main contributions are summarized as follows:

- To the best of our knowledge, RIDS is the first one that detects sensor and actuator attacks in nonlinear mobile robots subject to stochastic noises.
- A unique feature provided by RIDS is that it can detect sensor and actuator attacks without resorting to majority voting or Byzantine thresholds.
- Beyond detection, RIDS is capable of pinpointing attack targets and quantifying attack vectors. The information facilitates intrusion forensics and response.
- We implement a RIDS for Khepera ground mobile robot and evaluate the RIDS against various concrete attacks. Results indicate that as long as at least 1 out of 3 sensors on Khepera is uncorrupted, the evaluation shows less than 1% of false positive and false negative detection rate on average. Detection delay remains within an average of 0.40s.

2 OVERVIEW

This section describes the background for general mobile robots and the threat model considered in the paper. For succinctness reason, mobile robots are referred to as robots in the remaining of the paper.

2.1 Sensing and Actuation Workflow

Figure 1 shows a general system model for robots. It consists of a robot platform and a planner. The robot interacts with the physical

world through sensors and actuators in its physical-layer. Robot cyber-layer runs programs including device drivers, utility processes that perform data processing or translation, etc. We define each sensing procedure from capture of physical signal (e.g., electromagnetic waves, acoustic waves), signal digitization, data processing, to data transmission to the planner as a *sensing workflow*. Analogously, we define the counterpart procedure that receives, translates and executes control commands in an actuator as an *actuation workflow*. Figure 1 reflects the system model of many real-world robots, such as MIT autonomous car [36] and Tartan Racing robot [58].

For extensibility and security purposes, recent advances in robot systems adopt modular design principle instead of bulky integration. The development of embedded systems shows a trend of running different tasks of a robot system on separate mission-specific micro-processing chips. For instance, a modern car integrates more than 100 ECUs virtually into every functioning and diagnostics aspect [32]. Microkernels are extensively supported and employed in embedded systems [16, 59] to keep device drivers and applications isolated by a secure layer. Given these popular design patterns, we model that each sensing workflow or actuation workflow, i.e., device drivers and utility processes, runs in isolation with each other.

The planner is the control center of a robot. Because of the security and robustness significance of the planner, separation is also enforced between the planner and the robot platform. For instance, the planner could run in a separate chip, or the trust-zone of a processor, or even reside in a physically remote location. It receives sensor readings and sends control commands to the robot platform using certain communication protocols (e.g., CAN) [18].

In what follows, we depict the attacker and the defender considered in this paper.

2.2 Threat Model

The attacker considered in this work can observe real-time robot states and has knowledge about robot actuators and sensors. The attacker can launch actuator attacks and/or sensor attacks on one or multiple sensing or actuation workflow(s) through different channels, including malware (e.g., logic bomb), signal interference (e.g., spoofing) or physical damage (e.g., wire cut-off).

Given the attacker model, our detection system *does not assume any* particular sensing workflow or actuation workflow to be trusted. We assume that an attacker could not compromise all sensing workflows and corrupt all sensor readings simultaneously. Under the design where workflows run with isolation (see Section 2.1), the attacker’s ability to compromise a workflow does not imply the ability to compromise another. To the best of the authors’ knowledge, no reported attack is capable of compromising all sensor workflows and tampering all sensor readings. Admittedly, such attacks could be possible; however, it poses enormous difficulty for attackers. Firstly, for heterogeneous sensors, holding a vulnerability and corresponding exploit targeted on one sensing workflow is costly for an attacker [48, 62]. Hence, it is tough to corrupt all sensors. Secondly, even if an attacker is capable of corrupting all sensors, the attacker needs to launch the attacks simultaneously to

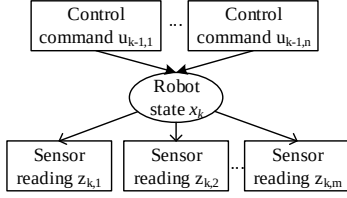


Figure 2: Correlation between control commands, robot states, and sensor readings.

avoid detection. It is a great challenge to launch such coordinated attacks on different target sensing workflows. [49].

The planner contains a defender that aims to detect sensor and actuator attacks targeted on the robot. Because of its security significance, the planner typically maintains minimal code complexity. Its code is extensively tested before deployment, and isolated from the rest of the system. We consider the planner as a trusted computing base (TCB), and the defender keeps no secret information from adversaries.

2.3 Our Approach

In robotics and control theory, *state* refers to the instantaneous description of a dynamical system which changes over time, e.g., the position and orientation of a vehicle, the pitch, yaw, and roll of a drone, etc. **Control algorithms utilize sensor readings to estimate system states and generate control commands for robot actuators.**

Key insight In mobile robots, control commands and sensor readings are correlated using robot states as an intermediate (shown in Figure 2). Specifically, executed control commands determine how the robot evolves from an initial state to a new state during a period of time. And the new state is captured by sensor readings. Sensor readings can be utilized to estimate new states. Executed control commands can be estimated through the comparison of the initial and new states. Hence, a discrepancy between planned control commands and executed control commands estimated by sensor readings indicates the existence of actuator attacks. Moreover, multiple sensors in a mobile robot typically have redundancy regarding their measured signals [12, 13, 21]. For instance, during a short period, a wheel encoder sensor measures the traveled distance by a wheel, and a LiDAR sensor measures distances between a robot and nearby obstacles. With the knowledge of the robot initial position and heading, both sensors can estimate the current position and heading. Because of sensor redundancy, the states estimated by different sensors could overlap, which can be utilized for detecting sensor attacks by cross-validation. Therefore, by comparing estimated control commands and planned control commands, we can detect actuator attacks. By comparing estimated states across sensors, we can detect sensor attacks. We develop a RIDS based on this key insight.

3 ROBOT FORMALIZATION AND PROBLEM STATEMENT

In this section, we formally model the general mobile robot system shown in Figure 1 and formulate our detection problem. We provide the high level intuition of our approach at the end.

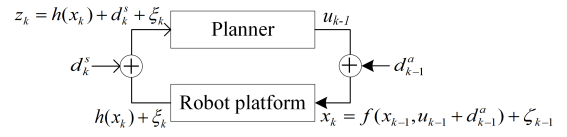


Figure 3: Robot formal modeling considering actuator attacks and sensor attacks.

3.1 Robot Formal Modeling

A mobile robot can be modeled as a nonlinear discrete time dynamic system. In each control iteration $k \in \{1, 2, \dots\}$, the planner generates planned control commands $\mathbf{u}_{k-1}$. After the commands being executed by robot actuators, the robot states evolve from $\mathbf{x}_{k-1}$ to $\mathbf{x}_k$. Under the new states, the planner receives new sensor readings $\mathbf{z}_k$. The system model can be formally described by the following equations:

$$\begin{aligned} \mathbf{x}_k &= f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \zeta_{k-1} \\ \mathbf{z}_k &= h(\mathbf{x}_k) + \xi_k. \end{aligned} \quad (1)$$

The first equation in (1) is referred to as the *kinematic model*, which describes robot state transitions caused by control commands. The kinematic model specifies the relation between states and control commands based on the actuator properties, e.g., how the actuators function, and where the actuators are located. For instance, a quadcopter's controller adjusts the speeds of the 4 rotors to maneuver itself, while a two-wheel differential drive robot sets different speeds of individual wheels to move along a straight line or take a turn. Function $f(\cdot)$ is referred to as the kinematic function.

The second equation in (1) is the *measurement model*, which describes the relations between sensor readings and robot states. The measurement model is determined by the robot sensor settings, such as sensors types, sensor placement, etc. Function $h(\cdot)$ is referred to as the measurement function. Vectors ζ_{k-1} represents process noises, which account for external disturbances in the kinematic model. Vectors ξ_k stand for measurement noises, which account for sensing inaccuracy. We assume noise vectors are Gaussian with zero mean and known covariances Q and R , respectively. Note that Gaussian noise approximation is common in control system modelings [33].

System (1) is general to model all nonlinear robots. Note that the system model is an essential requirement for control purposes during robot design phase. Hence, the modeling described in this section does not introduce extra burden to security administrators.

Sensor Attack tampers data in a sensor workflow and results in wrong sensor readings received by the planner. When sensor attack is launched, sensor readings $\mathbf{z}_k$ received by the planner can be modeled as:

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{d}_k^s + \xi_k \quad (2)$$

where $\mathbf{d}_k^s$ is the attack vector representing corruptions on authentic sensor readings. The robot is free of sensor attack when $\mathbf{d}_k^s = 0$. Corruptions might exist for multiple sensors. After sensor attacks occur, the control algorithm of the robot might be lured to generate erroneous control commands.

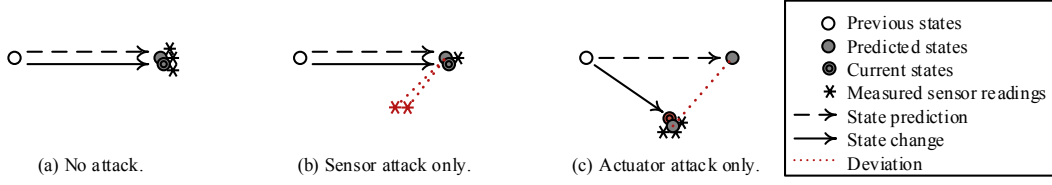


Figure 4: High-level intuition for sensor attack and actuator attack detection in robots. The detection is enabled by comparison between measured sensor readings and predicted states. Red circle and red star denotes corrupted states (actuator attack) and corrupted sensor readings (sensor attack), respectively.

Actuator Attack directly alters the control commands executed by the actuators in an actuation workflow. Considering actuator attacks, the kinematic model can be modeled as:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1} + \mathbf{d}_{k-1}^a) + \zeta_{k-1} \quad (3)$$

where $\mathbf{d}_{k-1}^a$ is actuator attack vector. The robot is free of actuator attack when $\mathbf{d}_{k-1}^a = 0$.

3.2 Problem Statement

Consider a robot as modeled in Figure 3 that receives sensor readings $\mathbf{z}_k$ from m sensing workflows and sends control commands $\mathbf{u}_{k-1}$ to n actuation workflows. An attacker could launch actuator attack by attack vector $\mathbf{d}_{k-1}^a$ and/or launch sensor attack by attack vector $\mathbf{d}_k^s$. The robot model with sensor and actuator attacks is:

$$\begin{aligned} \mathbf{x}_k &= f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1} + \mathbf{d}_{k-1}^a) + \zeta_{k-1} \\ \mathbf{z}_k &= h(\mathbf{x}_k) + \mathbf{d}_k^s + \xi_k \end{aligned} \quad (4)$$

In this work, we aim to detect the occurrence of sensor and/or actuator attacks in the robot. In addition, we intend to identify the specific workflow(s) $i \in \{1, 2, \dots, m\}$ and $j \in \{1, 2, \dots, n\}$ on which attacker targets, and quantify the attack vectors $\mathbf{d}_k^s$ and $\mathbf{d}_{k-1}^a$ as diagnosis information for future analysis.

4 HIGH-LEVEL INTUITION ON WHY MAJORITY VOTING FAILS AND OUR APPROACH WORKS

Under powerful attack scenarios when majority of sensors are corrupted, the IDS's perception of the physical status (e.g., position) of a robot could be greatly distorted. Majority-voting based approaches solely rely on measured sensor readings $\mathbf{z}_k$. In our approach, however, the IDS achieves the detection leveraging not only $\mathbf{z}_k$ but also physical dynamics. In particular, physical dynamics are leveraged to predict state evolution when attacks are absent. Deviations between such state predictions and measured sensor readings indicate occurrence of attacks.

To understand the intuition of our approach, we tentatively consider a robot with 3 sensors at a time instant $k - 1$. For ease of presentation, we tentatively do not consider measurement and process noises. After the execution of control commands, the robot will evolve from the previous states into the current states.

We consider the following possible attack conditions within one control iteration from $k - 1$ to k .

- When the robot is free of attack, the 3 measured sensor readings are consistent with each other as shown in Figure 4 (a). Majority-voting based approaches raise no alarm. In our approach, we firstly predict state evolution. Then we compare measured sensor readings with the predicted states. The consistency indicates that the robot is not under attack.
- When only sensor attack is launched, and 2 out of 3 sensors are corrupted (Figure 4 (b)), majority voting-based approaches regard the two sensors that are consistent with each other to be correct and the other one as corrupted. Hence, majority voting makes an obvious mistake here. In our approach, the predicted states serve as the ground truth, and the deviation between predicted states and the 2 measured sensor readings indicates sensor attack. Moreover, our approach can correctly tell which sensors are corrupted and which is not.
- When only actuator attack is launched (Figure 4 (c)), measured sensor readings reflect the actual current states and are consistent with each other, and majority voting-based approaches raises no alarm. On the contrary, in our approach, we notice deviations between the measured sensor readings and the predicted states. The deviations indicate existence of actuator attacks.

Based on the intuition, we present the design of proposed robot intrusion detection system in the next section.

5 ROBOT INTRUSION DETECTION SYSTEM DESIGN

We develop a robot intrusion detection system (RIDS) framework based on estimation theory for the detection of actuator attacks and sensor attacks. **RIDS runs inside the planner (see Figure. 1). The defender's goal is as follows: 1) detect actuator and sensor attacks; 2) identify the targets of detected attacks, i.e., which sensing/actuation workflows are attacked; 3) quantify the data corruptions of the detected attacks.** Figure 5 shows the schematic of RIDS. Algorithm 1 describes the step-by-step procedure of RIDS. The complete RIDS algorithm is described as Algorithm 2 in Appendix. Some notations used in the algorithm are explained in Table 1.

RIDS consists of four modules: a monitor, a multi-mode estimation engine, a mode selector, and a decision maker. RIDS runs iteratively from the start until the end of a mission. In each control iteration, the monitor firstly collects data and sends it to the estimation engine. The estimation engine generates a set of estimation results under different hypothesis and their corresponding likelihoods. Then the mode selector accepts the more likely hypothesis. Finally the decision maker leverages the estimation results from

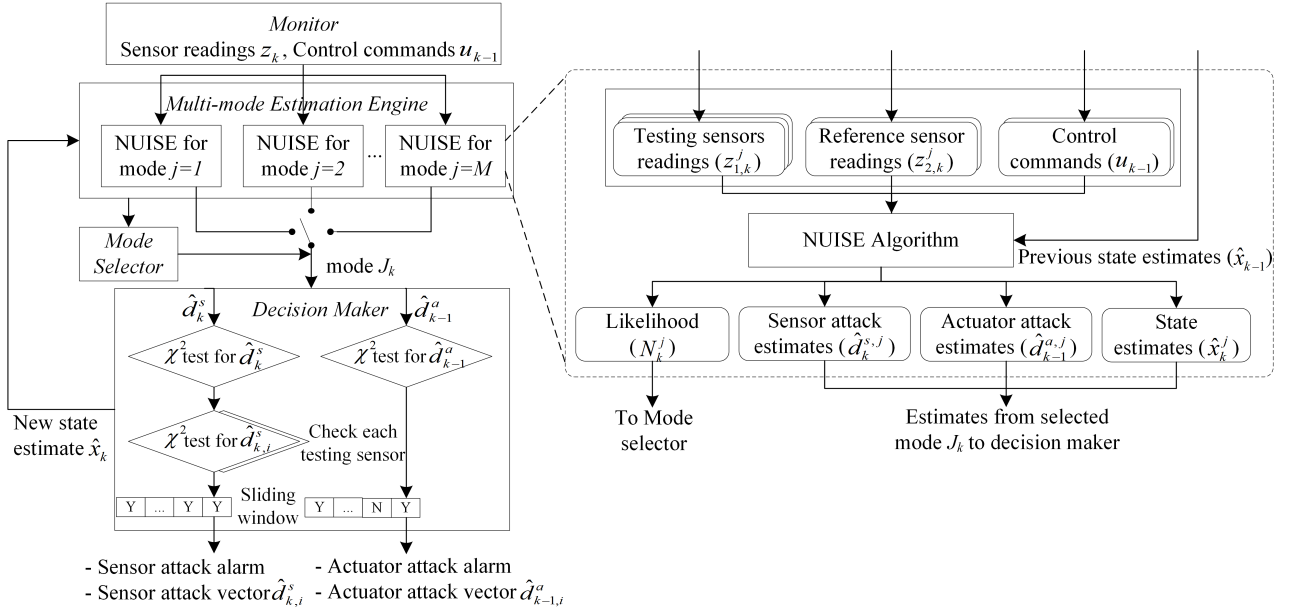


Figure 5: Robot intrusion detection system (RIDS) overview.

the accepted hypothesis to detect attacks. We explain the detail of each module in the sequel.

Table 1: Selected notations in Algorithm 1

Notation	Explanation
w_s/w_a	sliding window size for sensor/actuator attack
c_s/c_a	decision criterion for sensor/actuator attack
α_s/α_a	confidence level for sensor/actuator attack
M	different modes in RIDS
$z_{1,k}^j/z_{2,k}^j$	testing/reference sensor readings in mode j
N_k^j	likelihood of mode j

5.1 Monitor

In each control iteration, control module delivers a copy of control commands u_{k-1} to the monitor of RIDS (Algorithm 1 line 2). After control commands execution, the monitor collects sensor readings from all onboard sensors z_k through the communication module (line 3). The monitor sends the received data to the multi-mode estimation engine.

5.2 Multi-mode Estimation Engine

The goal of the estimation engine is to obtain minimum variance unbiased estimates for actuator attack vectors d_{k-1}^a and sensor attack vectors d_k^s in order to determine attack occurrences. Minimum variance unbiased estimates require that the expected value of estimates should equal to their corresponding target value, and the estimation error variance must be minimized. To achieve this goal, we use robot state estimates $\hat{x}_k$ as an intermediate, and obtain the attack vector estimates leveraging the correlation between robot

states, sensor readings, and control commands as shown in Figure 2. However, the estimation engine faces several challenges.

Challenge 1: Majority of sensors could be potentially corrupted, and we have no knowledge about which sensor(s) is(are) corrupted. Using corrupted sensor readings would result in wrong state and attack vector estimates.

Challenge 2: Existing work does not consider actuator attacks, and directly use planned control commands for state prediction. Under actuator attacks, executed control commands deviate from planned control commands. Since executed control command cannot be directly monitored from the physical world, estimation in the presence of actuator attacks is challenging.

Challenge 3: Real-world robots are nonlinear systems, and they are rooted with inaccuracies in sensing and actuation. It is challenging to build a RIDS which can detect attacks on nonlinear system subject to noises.

To address challenge 1, we propose a multi-mode estimation engine that calculates estimates along with the likelihoods of possible attack conditions. In particular, the multi-mode estimation engine maintains a set of possible sensor attack conditions. Each condition is referred to as a *mode*, which represents a hypothesis that a particular subset of sensors is potentially attacked, and remaining sensors are clean. The potentially attacked sensors are referred to as *testing sensors*, and the clean sensors are referred to as *reference sensors*. Each mode runs a nonlinear unknown input and state estimation (NUISE) algorithm in parallel (line 4-7). Leveraging the reference sensor readings and planned control commands from the last iteration, NUISE estimates new robot states, corruptions on testing sensor readings, corruptions on control commands, and a likelihood for each mode.

NUISE algorithm The NUISE algorithm is described in Figure 6. At control iteration $k-1$, the algorithm predicts the states at next

Algorithm 1 Robot Intrusion Detection System (RIDS)

Input: Initial state estimates $\hat{\mathbf{x}}_{0|0}$; robot kinematic function $f(\cdot)$; measurement function $h(\cdot)$; parameters $w_s, w_a, c_s, c_a, \alpha_s, \alpha_a$;

Output: Detection decision; attack vector estimates

```
1: for control iteration  $k \leftarrow 1$  to  $\infty$  do
2:   Receive control commands  $\mathbf{u}_{k-1}$ ;
3:   Receive sensor readings  $\mathbf{z}_k$ ;
4:   for mode  $j = 1$  to  $M$  do
5:     Run NUISE with input  $(\mathbf{u}_{k-1}, \hat{\mathbf{x}}_{k-1|k-1}, \mathbf{z}_{1,k}^j, \mathbf{z}_{2,k}^j)$ , and
     generate  $(\hat{\mathbf{x}}_{k|k}^j, \hat{\mathbf{d}}_k^{s,j}, \hat{\mathbf{d}}_k^{a,j}, \mathcal{N}_k^j)$ ;
6:      $\mu_k^j \leftarrow \max\{\mathcal{N}_k^j \mu_{k-1}^j, \epsilon\}$ ;
7:   end for
8:   Select mode  $J_k \leftarrow \text{argmax}_j \text{normalized } \mu_k^j$ ;
9:   Obtain state estimates  $\hat{\mathbf{x}}_{k|k}^{J_k}$ , sensor attack estimates  $\hat{\mathbf{d}}_k^{s,J_k}$ ,
   and actuator attack estimates  $\hat{\mathbf{d}}_k^{a,J_k}$  from  $J_k$ ;
10:   $b_k^s \leftarrow$  Chi-square test on sensor attack;
11:   $b_k^a \leftarrow$  Chi-square test on actuator attack;
12:  if  $b_k^s = \text{True}$  and sliding window condition met then
13:    for each testing sensor  $t$  in mode  $J_k$  do
14:      Split sensor attack vector estimates  $\hat{\mathbf{d}}_{k,t}^{s,J_k}$  from  $\hat{\mathbf{d}}_k^{s,J_k}$ ;
15:      if Chi-square test on  $\hat{\mathbf{d}}_{k,t}^{s,J_k} = \text{True}$  then
16:        Confirm sensor attack on sensor  $t$ ;
17:      end if
18:    end for
19:  end if
20:  if  $b_k^a = \text{True}$  and sliding window condition met then
21:    Confirm actuator attack;
22:    for each actuator  $t$  in all do
23:      Split actuator attack vector estimates  $\hat{\mathbf{d}}_{k-1,t}^{a,J_k}$  from
       $\hat{\mathbf{d}}_{k-1}^{a,J_k}$ ;
24:    end for
25:  end if
26: return Confirmed attack(s); sensor attack estimates  $\hat{\mathbf{d}}_{k,t}^{s,J_k}$  ( $t \in$ 
  {testing sensors in mode  $J_k$ }); actuator attack estimates  $\hat{\mathbf{d}}_{k-1,t}^{a,J_k}$ 
  ( $t \in \{1, \dots, n\}$ );
27: end for
```

iteration using current state estimates $\hat{\mathbf{x}}_{k-1|k-1}$ and planned control commands $\mathbf{u}_{k-1}$. The predicted states should reflect a match with the reference sensor readings $\mathbf{z}_{2,k}$ in each mode. Whenever a deviation is detected between $\mathbf{z}_{2,k}$ and the reflected readings, actuator attacks can be detected (Step 1). With the identified actuator attack estimates from step 1, we conduct a new state prediction with corrected control commands $\mathbf{u}_{k-1} + \hat{\mathbf{d}}_{k-1}^a$ (Step 2). Then the predicted states is corrected by reference sensor readings $\mathbf{z}_{2,k}$, and we obtain the state estimates $\hat{\mathbf{x}}_{k|k}$ (Step 3). Finally, sensor readings reflected by the state estimates should match all sensor readings, and the deviations between that and testing sensor readings result in the detection of sensor attacks $\hat{\mathbf{d}}_{k,t}^s$ (Step 4). The full NUISE algorithm is presented as Algorithm 3 in Appendix.

When a mode is not consistent with the actual attack condition, i.e., corrupted sensors are falsely trusted, reference sensor readings

would have a larger discrepancy with state prediction. Subsequently, the state prediction in step 2 cannot be correctly compensated using the actuator attack estimates from step 1. NUISE leverages this discrepancy to generate a likelihood inverse proportional to the discrepancy.

It is a noteworthy point that the proposed detection algorithm does not base on voting mechanism. Even when a majority of sensor readings are corrupted, NUISE generates a higher likelihood for the mode that reflects the ground truth, independent of the number of testing/reference sensors in the mode. The number of modes to be tested grows with the number of sensors in a robot. More information on how to select the mode set is discussed in Section 7.

Challenge 2 is also addressed in NUISE algorithm. Using previous state estimates, planned control commands, and reference sensor readings, we calculate the actuator attack vector estimates $\hat{\mathbf{d}}_{k-1}^{a,J_k}$ (Step 1). We compensate the actuator attack vector estimates into the state prediction step (Step 2) to obtain unbiased state prediction.

In order to address challenge 3, we model noises with error covariance matrices. The matrices (i.e. noise models) propagation are tracked during each calculation step for all estimation results (see Algorithm 3 in Appendix A.2). The matrices serve two purposes: 1) minimizing the variances of the estimates during the estimation process; 2) normalizing attack vector estimates for hypothesis tests. In terms of the nonlinearity of the system, we incorporate nonlinear kinematic and measurement models to minimize estimation error, and use their linearized models to obtain minimum variance estimates. Notice that linearization is performed at the states and controls of each iteration.

5.3 Mode Selector

After a normalization, the mode selector compares the likelihood of each mode μ_k^j , and selects the mode J_k with the highest likelihood (line 8). The state and attack vector estimates of the selected mode J_k (line 9) will be leveraged for the decision-making process as follows.

5.4 Decision Maker

Using the attack vector estimates $\hat{\mathbf{d}}_{k-1}^a$ and $\hat{\mathbf{d}}_k^s$, the decision maker conducts Chi-square test to check whether estimated sensor and actuator attack vectors exceed the threshold under a certain level of confidence (line 10-11). In order to reduce the impact of transient fault during the mission, e.g., uneven ground or bump, etc., testing results go through a sliding window and RIDS raises alarm only when a certain number of positives appear in consecutive iterations (line 12 and line 20).

When the number of sensor attack positives exceeds the decision criteria c_s , RIDS raises sensor attack alarm. To further confirm that testing sensors are under attack, we separate the sensor attack estimates and conduct Chi-square test separately for an individual testing sensor (line 13-18). RIDS reports the confirmed sensors and their corresponding sensor attack vector. Analogously, RIDS raises actuator attack alarm, when the actuator attack positives exceed decision criteria c_a (line 21). RIDS calculates actuator attack vector estimates for each actuator $\hat{\mathbf{d}}_{k-1,i}^a$. Note that RIDS does not conduct Chi-square test on an individual actuator attack. Instead, it only

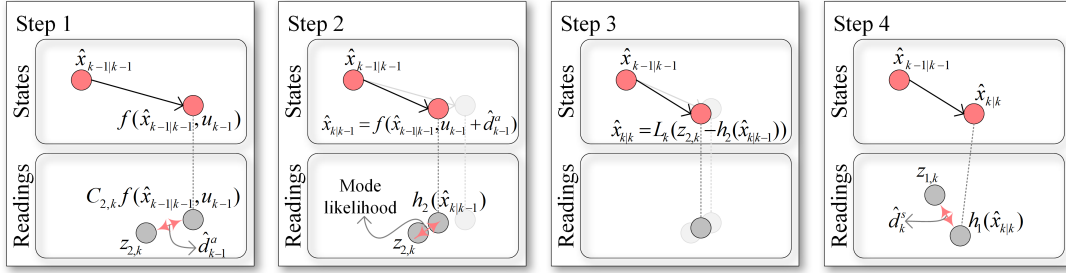


Figure 6: Nonlinear unknown input and state estimation algorithm execution. Step 1: actuator attack estimation. Step 2: State prediction with compensation. Step 3: State estimation. Step 4: Testing sensor attack estimation.

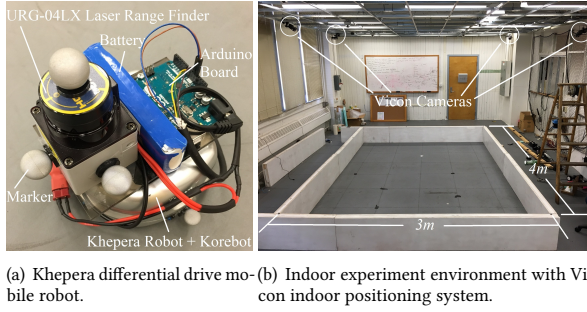


Figure 7: Khepera robot testbed and indoor positioning system.

checks the aggregated test statistics of actuator attack (explained in Appendix A.4).

Finally, the decision maker reports confirmed attack(s), sensor attack estimates, and actuator attack estimates to the security administrative as output.

6 EVALUATION

To understand the detection effectiveness and efficiency of RIDS for real-world robots, we implement RIDS on a Khepera mobile robot testbed, and conduct experiments under multiple attack scenarios. In this section, we first introduce the testbed and the mission. Then we describe the experiment setups and attack scenarios launched against Khepera. We analyze the detection results and discuss key parameter selection at last.

6.1 Robot Platform and Mission

Figure 7(a) shows an image of the robot system. It consists of Khepera III [3] differential drive robot mounted with KoreBot II [4] extension chip. Khepera is actuated by two wheels on its chassis. KoreBot runs OpenEmbedded Linux, which enables in-robot programming and control. The robot is equipped with three sensors: a wheel encoder, a laser range finder (LiDAR), and an indoor positioning system (IPS). The wheel encoder calculates the traveled distance of each wheel in a short period. Given its previous state, the traveled distance is further processed into its current position

and orientation. LiDAR scans laser beams in 240 degrees and receives reflection to obtain distances from surrounding objects. IPS is powered by Vicon motion capturing system (see 7(b)), which tracks the position of the robot. The Kinematic model of Khepera and the measurement models of the three sensors are described in Appendix A.3.

Mission We conduct a motion planning mission where the robot is steered from an initial location to a target location. It avoids collisions with some obstacles on its path. The mission proceeds as follows: 1) Before the mission starts, Khepera receives map information containing the environment setup (obstacles and wall boundaries) and the target location. 2) The planner calculates a collision free path using optimal rapidly-exploring random trees (RRT*) algorithm [28]. 3) The robot executes PID closed-loop control [51] to track the planned path using real-time positioning data from IPS.

6.2 Experiment and Attack Setups

Evaluation setup For comparison purpose, we use an identical path generated from RRT* for all scenarios in the experiments. In each experiment, Khepera travels from a starting point at $(0m, -1.2m)$ to a target point $(0m, 1m)$ inside a $3m \times 4m$ confined space shown in Figure 7(b), with constant 7000 speed units¹. Three $0.8m \times 0.2m \times 0.2m$ cube-shaped obstacles are on the ground between the starting and the target location. RRT* algorithm generates a path that avoids the obstacles, and Khepera follows the path using PID ($P = 0.8, I = 0, D = 0.001$) control. We identify measurement noise covariance R and the process noise covariance Q by referring to the data sheets of the sensors along with some empirical experiments (refer to [6] for more systematic approaches). RIDS generates detection results under confidence level of 0.05 for actuator attacks, and 0.005 for sensor attacks. We choose 2 positives out of 2 windows as the decision criteria for sensor attacks, and choose 3 positives out of 6 windows as the decision criteria for actuator attacks. We will justify how these configurations are chosen in Section 6.6 by evaluating RIDS across a range of different parameters.

Attack setup We conduct multiple attack scenarios during the mission as described in Table 2. We intend to demonstrate that RIDS works well regardless of the attack channels or sensor/actuator targets on the robot. The attack scenarios target on different sensing

¹Speed ratio 144010 units per m/s , 7000 units is approximately $0.05m/s$.

Table 2: Attack scenarios launched against Khepera mobile robot.

Attack Scenario	Attack Scenario Description
Wheel controller logic bomb	Logic bomb in the actuator utility library that alters the control commands to be executed
Wheel jamming	Physically jamming a particular wheel so that the wheel sticks
IPS logic bomb	Logic bomb in the IPS data processing library that alters authentic positioning data
IPS spoofing	Fake IPS signal that overpowers authentic source and sends out fake positioning data
Wheel encoder logic bomb	Logic bomb in the wheel encoder data processing library that alters readings
LiDAR sensor blocking	Blocking laser ejection and reception in particular angles of the LiDAR
LiDAR DOS	Denial of service by cutting off the LiDAR sensor wire connection
Wheel controller and IPS logic bombs	Altering both wheel control commands and IPS readings through logic bombs
LiDAR DOS and wheel encoder logic bomb	Blocking LiDAR readings and altering wheel encoder readings
IPS spoofing and LiDAR DOS	Altering IPS readings and blocking LiDAR readings
IPS and wheel encoder logic bombs	Altering both IPS and wheel encoder readings through logic bombs

or actuation workflows of the robot, and launch actuator and sensor attacks through various channels including cyber and physical channels. We inject several logic bombs into the data processing libraries of the IPS and the wheel encoder. The logic bombs can be triggered at a particular time after the mission starts, and continuously alter the authentic sensor readings afterward. For instance, we can trigger the logic bomb to stealthily shift the positioning data received from IPS by a certain distance along the X axis. A logic bomb is also injected into the wheel controller library to alter control commands for the two wheels. Wheel jamming attack is launched by physically jamming a wheel so that the wheel stops moving. IPS spoofing attack is launched by overriding authentic IPS signals from the Vicon system and sending fake positioning data, analogously to GPS spoofing attacks. For LiDAR, we launch sensor attack by blocking the signal ejection and reception channel in particular directions. Besides, we launch the attack that sabotages the signal transmission by physically cutting off its wire connection. To evaluate the RIDS when multiple sensing workflows or actuation workflows are under attack, we launch several attack scenarios where several of the aforementioned attacks are combined. Table 3 shows quantitative information about the details of the attack scenarios. In addition to attack scenarios, we also conduct 9 scenarios under which the mission finishes without attack.

6.3 Detection Effectiveness

RIDS aims at detecting, identifying, as well as quantifying attacks in robots. To evaluate the effectiveness of RIDS, we define *true positive* as a time instant that 1) raises alarm if the robot is under attack, and 2) identifies the correct sensor/actuator attack condition, i.e., which sensing or actuation workflow is attacked. Otherwise, positive detection result is considered as *false positive*. *False negative* is defined as a time instant when RIDS does not raise alarm when any workflow is under attack. If all workflows are free of attacks and RIDS does not raise any alarm, the time instant is referred to as *true negative*. The detection result column in Table 3 shows identification of attack type and attack condition for different scenarios. From the 11 attack scenarios, we observe that both types of attacks launched from different channels can be successfully detected and identified. Scenario #1, #2 and #8 involves actuator attacks launched from different channels. The robot is under both actuator and sensor

attack under scenario #8. Under scenario #8, #9 and #10, 2 out of 3 sensors on the robot are corrupted and only one sensor remains uncorrupted.

For the ease of presenting classification results, Table 4 defines the possible attack conditions for actuator and sensor attacks. We refer to these attack conditions as sensor modes and actuator modes. Figure ?? presents graphical details of the detection results for several attack scenarios. Each figure includes eight plots that depicts the outputs from RIDS: 1) IPS sensor attack vector estimates ($\mathbf{d}_{k,I}^s$); 2) wheel encoder sensor attack vector estimates ($\mathbf{d}_{k,W}^s$); 3) LiDAR sensor attack vector estimates ($\mathbf{d}_{k,L}^s$); 4) actuator attack vector estimates for the wheels ($\mathbf{d}_k^a$); 5) sensor attack Chi-square hypothesis test statistic and threshold under confidence level $\alpha = 0.005$; 6) sensor mode selection; 7) actuator attack Chi-square hypothesis test statistic and threshold under confidence level $\alpha = 0.05$; 8) actuator mode selection. Figure ?? shows a scenario when wheel controller control commands and IPS sensor readings are tampered by logic bombs at different time instants. Around 4s, IPS sensor attack vector estimates on the X axis surge (plot 1). Accordingly, sensor attack test statistic surges above the threshold (plot 5), and sensor mode selection (plot 6) indicates that the robot is under IPS sensor attack. Around 10s, actuator attack vector estimates on the left and right wheel significantly deviate from 0. Accordingly, we notice an oscillating surge over the threshold for actuator attack (plot 7), and actuator mode selection (plot 8) indicates that the robot is under actuator attack. Throughout the experiment, both sensor attack estimates for wheel encoder and LiDAR remain silent. Figure ?? shows a scenario where attacks against multiple sensors are launched/revoked at four different time instants. We observe that the detection results are highly consistent with the attack scenario. Detection results for some other scenarios can be found in Appendix A.6.

We examine the false positive and false negative time instants occurred in the experiments. Majority of false classifications are introduced by the sliding window for the purpose of transient fault tolerance. False positives and false negatives are inevitable at the edge when an attack becomes active or revoked, and the choice

²False positive rate and false negative rate.

³Subscript $i \rightarrow j$ stands for transition from sensor/actuator mode i to mode j . W, I, and L stands for wheel encoder, IPS, and LiDAR, respectively.

Table 3: Attack scenarios and detection results from RIDS.

#	Attack Scenario	Launch Time (s)	Attack Type (Channel)	Attack Description	Detection Result	Detection Delay (s)	FPR/FNR ²
1	Wheel controller logic bomb	16.0	Actuator (cyber)	-6000 units on v_L +6000 units on v_R	$A_{0 \rightarrow 1}$ ³	0.49	A: 0 / 0.83% S: 1% / -
2	Wheel jamming	5.3	Actuator (physical)	0 unit on v_L	$A_{0 \rightarrow 1}$	0.76	A: 0 / 3.1% S: 0 / -
3	IPS logic bomb	19.0	Sensor (cyber)	shift +0.07m on X	$S_{0 \rightarrow 1}$	0.30	A: 0 / - S: 1.6% / 0.24%
4	IPS spoofing	26.0	Sensor (physical)	shift -0.1m on X	$S_{0 \rightarrow 1}$	0.24	A: 2.24% / - S: 1.55% / 1.39%
5	Wheel encoder logic bomb	16.0	Sensor (cyber)	increment 100 steps on left wheel encoder	$S_{0 \rightarrow 2}$	0.43	A: 1.4% / - S: 0 / 0.45%
6	LiDAR DOS	0.0	Sensor (physical)	received distance reading is 0m in each direction	S_3	0.23	A: 0 / - S: 0 / 0
7	LiDAR sensor blocking	7.0	Sensor (physical)	received distance reading to the left wall is incorrect	$S_{0 \rightarrow 3}$	0.55	A: 0.22% / - S: 0 / 0.80%
8	Wheel controller & IPS logic bomb	W: 10.0 I: 3.8	Sensor&Actuator (cyber)	∓ 6000 units on v_L, v_R shift +0.07m on X	$A_{0 \rightarrow 1}$ $S_{0 \rightarrow 1}$	W: 0.59 I: 0.50	A: 0 / 1.8% S: 0 / 0.24%
9	LiDAR DOS & wheel encoder logic bomb	W: 16.0 L: 25.0	Sensor (cyber&physical)	increment 100 steps on left wheel 0m in each direction from LiDAR	$S_{0 \rightarrow 2 \rightarrow 4}$	W: 0.43 L: 0.29	A: 0 / - S: 0.48% / 0.72%
10	IPS spoofing & LiDAR DOS	L: 10.0 I: 17.0 L: 25.0	Sensor (physical)	0m in each direction from LiDAR shift +0.07m on X LiDAR readings are restored to normal	$S_{0 \rightarrow 3 \rightarrow 5 \rightarrow 1}$	L: 0.36 I: 0.29 L: 0.30	A: 0.25% / - S: 0.25% / 0.58%
11	IPS & wheel encoder logic bomb	W: 10.0 I: 28.0	Sensor (cyber)	increment 100 steps on left wheel shift +0.1m on X	$S_{0 \rightarrow 2 \rightarrow 6}$	W: 0.33 I: 0.31	A: 0 / - S: 0.25% / 0.33%

Table 4: Sensor and actuator mode definition.

Sensor Mode #	Robot Attack Condition
S_0	under no sensor attack
S_1	under IPS sensor attack
S_2	under wheel encoder sensor attack
S_3	under LiDAR sensor attack
S_4	under wheel encoder and LiDAR sensor attack
S_5	under IPS and LiDAR sensor attack
S_6	under IPS and wheel encoder sensor attack
Actuator Mode #	Robot Attack Condition
A_0	under no actuator attack
A_1	under actuator attack

of window size and decision criteria determines the number of false classifications. For sensor attack false positives, we observe only a small portion is caused by sensor or actuator mode selection errors, while the majority is caused by bogus test statistics increases. The average false positive rate and false negative rates are 0.86% and 0.97%, respectively. Therefore, we believe the RIDS can be considered as effective in detecting and identifying both actuator attacks and sensor attacks targeted on our testbed.

6.4 Detection Delay

Detection delay indicates the time between when an attack is launched/revoked, and when RIDS captures the change. Theoretically, in each control iteration, attack vectors can be revealed in the very next iteration after launch. However, we add a sliding window in the decision maker to eliminate transient fault impact. Hence, detection delays will depend on the parameter choice. In our experiment, we choose 2/2 and 3/6 as the decision criteria and sliding window size. The detection delay for each attack scenario is shown in Table 3. We observe that the detection delays are quite small. Specifically, average detection delay for sensor attacks is 0.35s, and the counterpart for actuator attacks is 0.61s. The average delays are consistent with our parameter selection for actuator and sensor attacks. Through our analysis of the detection statistics, we notice that the test statistics raises above the threshold mostly in the next iteration after an attack occurs. Most delays are incurred by the sliding window decision making.

Once the magnitude of an attack exceed predetermined threshold, the maximal detection delay is a constant multiple of control iterations. The frequency of the control iteration is determined by hardware configurations (e.g., CPU frequency) and control algorithm design, which is chosen to meet the specifications of robots and mission requirement. Fast moving robots have higher frequency of control cycles, hence the detection delay would be small.

6.5 Attack Vector Quantification

Actuator attack and sensor attack vector estimates provide quantitative information about the attacks, which can assist security administrative for further diagnosis and attack response. For instance, after sensor attack detection in scenario #9, IPS sensor attack vector estimates on X axis $d_{k,I}^{s,x}$ is $+0.069m$ with a standard deviation of $\pm 0.002m$. Average error between estimated vector and the ground truth ($+0.07m$) is 1.91%. After actuator attack detection, average actuator attack vector estimates on the left wheel and right wheel are $d_k^{a,L} = -5975.4 \pm 1188$ units and $d_k^{a,R} = +5892.4 \pm 1091$ units, respectively. Average error between the estimated vector and the ground truth (∓ 6000 units) are 0.41% and 1.79%, respectively. We observe that the estimation results are fairly accurate for both actuator and sensor attack vector estimates.

6.6 Parameter selection

We evaluate the detection effectiveness of RIDS across different choices of detection window sizes (w), detection criteria (c), and detection confidence level (α) in detection of actuator and sensor attacks. The analysis is conducted over the 20 experiments including 11 different attack scenarios and 9 no-attack scenarios. Figure ?? depicts the ROC curve for actuator attack detection under different confidence levels range from $\alpha = 0.0005 \sim 0.995$. From the figure, we notice that the detection achieves an acceptable performance when $\alpha = 0.05$ under different w and c settings. The selection of w and c eliminates the impact of faults during the mission and determines whether a positive time instant should be regarded as an attack. with a chosen α , Figure ?? depicts the detection performance under different w and c . The results indicate that under certain window size, detection performance increases first and reduces afterward. We select $c/w = 3/6$ as the configuration, which yields the best performance. Analogously, we select $\alpha = 0.005$ as the optimal confidence level, and $c/w = 2/2$ as the optimal decision criteria/window size configuration for sensor attack detection.

6.7 Evasive attacks

An attacker's ideal goal is to bypass the detection of RIDS, yet be capable of causing significant impact to the robot or the environment it operates in. We can think of two possible ways of crafting such evasive attacks: 1) reduce attack vectors so that the test statistics in RIDS do not raise alarms; 2) frequently switch attack targets so that sliding window will treat the attack vectors as faults. Under the current RIDS configuration (α , w , c and sensor accuracy) in our experiments, the attack vector needs to be extremely small to remain alarm silence. For instance, we find that the distance shift for IPS sensor attack needs to remain under $0.02m$ to avoid detection. The speed alteration needs to remain under 900 units ($0.006m/s$) to avoid detection. Moreover, the control algorithm ensures that attack impact does not accumulate as time goes. Hence, we believe that an attacker cannot make a significant impact with the first approach. Since we demonstrate that the detection delay is small in Section 6.4, the impact of the second attack cannot succeed either.

7 DISCUSSION

In this section, we discuss some issues related to applying the RIDS to real-world robots.

General applicability RIDS is applicable to nonlinear systems, which covers majority of real-world complicated robot systems, such as UAV [43] and quadrotor [35]. The design and implementation of RIDS only require the kinematic model and the measurement model. In fact, both models are the essential requirements for any robot mission. Therefore, the RIDS incurs little extra mathematical modeling burden for security administrative. We present how RIDS can be applied to UAV as another type of nonlinear system in Appendix A.5.

Mimicry attacks Admittedly, RIDS cannot handle all active attacks targeted on robots. The attacker might carefully craft attacks vectors such that the mode probability of an incorrect mode be large as that of the true mode, for all the time. If this happens, RIDS can detect attacks as long as one sensor is clean, but might not be able to identify the correct attack vectors. Consider the case that the attacker launch mimicry attacks but at least one sensor is clean. If the mode estimator chooses incorrect mode, the actuator attack estimates would be incorrect since corrupted sensor is used as a reference sensor, but RIDS would notice that physical dynamics with incorrect actuator attack estimates are inconsistent with the testing sensor reading (uncorrupted sensor). Thus, RIDS will raise the alarm, although the attack vector estimates remain incorrect. If the mode selector chooses the correct mode, then RIDS estimates correct attack vectors as we explained beforehand. It is noteworthy that launching mimicry attacks requires more knowledge and computational power for the attacker, because the attacker should consider the influence of attacks on physical dynamics.

Noise, fault, and attack RIDS models measurement noises and process noises of a robot and estimates data corruptions with tracked noise propagation. Under certain confidence level, RIDS would not raise alarm under the influence of noises. In this paper, data corruptions model the effects of actuator and sensor attacks. In fact, unintentional actuator and sensor fault/malfunctioning may also result in the detection of data corruptions. From security and safety perspective, both fault and attack may thwart mission execution, and RIDS conducts the detection without distinguishing the two cases. Approaches to distinguish faults and attacks can leverage statistical or knowledge-based fault modeling [14, 45], which is beyond the scope of this paper. For the attacks that can be detected and identified, RIDS cannot distinguish different attack channels which result in the same attack vectors.

Mode set selection In the multi-mode estimation engine design, each mode represents a hypothesis that particular reference sensors are clean and the rest of testing sensors are potentially corrupted. The number of modes M grow linearly with the number of onboard sensors in a robot, and the computational complexity grows accordingly. The choice of M is a trade-off between computational complexity and detection accuracy. In particular, with m sensing workflows, the number of possible attack conditions grows exponentially where $M_{complete} = 2^m - 1$ (exclude the condition when all sensors are corrupted). Noticeably, the mode set that assumes only one reference sensor remains the same detection capability with that which considers $M_{complete} = 2^m - 1$ as demonstrated in [30]. Hence, we employ the current mode set in the multi-mode estimation engine in favor of computational complexity. In the NUISE algorithm of each mode, reference sensor

readings are leveraged in the estimation process, and their sensing variances are propagated into the the state and attack vector estimates (see the propagation of $R_{z,k}^j$ in Algorithm 3). When multiple reference sensors are available in a mode, the estimation process can perform *sensor fusion* [52] and reduce estimation variances. Hence, adding more modes into the estimation engine increases the detection accuracy when multiple sensors remain uncorrupted. Table 5 shows actuator attack vector variance comparison in different modes. Noticeably, the mode which assumes all sensors are uncorrupted generates lowest variance.

Table 5: Actuator attack vector quantification variance under different mode.

Reference Sensor(s)	Var on $V_l (\times 10^{-5})$	Var on $V_r (\times 10^{-5})$
IPS	2.39	1.94
Wheel encoder	2.76	2.04
LiDAR	21.7	20.3
all sensors	2.32	1.88

Sensor capability This paper only considers mobile robot states in terms of movements. Hence, sensors that measure other statuses of robots, e.g., temperature, tire pressure, are out of scope. During the estimation process, the NUISE Algorithm estimates robot states using reference sensor readings in each mode. A requirement is that the reference sensor(s) in each mode are capable of reconstructing all robot states in a control iteration. However, some sensors might not be utilized to reconstruct all states of a robot. For instance, consider a ground robot equipped with a magnetometer which can only measure the orientation θ_M of the robot. Since robot states are described as (x_k, y_k, θ_k) , the measurement from magnetometer cannot reconstruct the robot states. If the robot runs RIDS, then the mode that only takes magnetometer as reference sensor will fail to estimate the states and the attack vectors. Under such cases, RIDS designers can group multiple sensors together to ensure the reference sensors of each mode can reconstruct robot states. For instance, the magnetometer can be grouped together with a GPS sensor and use $\mathbf{h}(\mathbf{x}_k) = [x_k, y_k, \theta_k, \theta_k]^T$ as their joint measurement model.

8 RELATED WORK

The security of robots and other cyber-physical systems (CPS) has been attracting increasing attention. In this section, we review some preliminary studies concerning several topics related to this work.

Existing attacks on robots Preliminary works identified attacks launched through different channels, including physical damage, network communication, signal interference, malware, etc. Koscher et al. demonstrated that virtually any ECUs inside the internal vehicular network of a modern vehicle can be infiltrated through physical access [32]. Checkoway et al. further demonstrated that remote exploitation through wireless channels, such as Bluetooth or cellular radio, is also possible [9]. AnonSec group took over a NASA Global Hawk drone and tried to crash the drone into the ocean by breaking into internal network [54]. Several studies [23, 57, 67] investigated spoofing attacks targeted on civilian GPS signals. Some researchers have also implemented deceptive

spoofers and conducted proof of concept attack experiments [23, 42]. Son et al. [56] demonstrated that resonant frequency of sound could be used to incapacitate a drone through its gyroscope sensor. Although at an early development stage, robot malware has already debuted. Sasi [29] developed a backdoor program which allows attackers to control the drone remotely.

Intrusion detection for CPSs State estimation theory has been utilized to detect sensor attacks for linear cyber-physical systems in recent works [7, 41, 44, 45]. Several works [20, 46, 64] study both actuator and sensor attacks for linear cyber-physical systems with estimation theory. In contrast, most real-world robots are modeled as nonlinear systems, such as Khepera and UAVs. In [20, 44–46], processing and measurement noises rooted in actuators and sensors are not considered or considered with bounded support. In contrast, real-world robots are subject to stochastic noises with unbounded support. Shoukry et al. [55] proposed a sensor attack detection approach against signal interference attacks by verifying randomly inserted probes. A few studies in sensing systems proposed GNSS attack detection techniques [42, 50]. Montgomery et al. proposed to detect GNSS attacks by exploiting the effects of intentional high-frequency antenna motion [42]. Psiaki et al. validated the correctness of civilian GPS signals using dual-receiver correlation of military signals [50]. Some of these techniques require homogeneous sensors or extra hardware to enable a comparison between sensors, and some require cryptography for authentication purposes. In robot systems, sensors usually measure different physical signal configurations. Extra hardware brings additional costs and burdens for power supply and weight carrying.

9 CONCLUSION AND FUTURE WORK

Sensor attacks and actuator attacks targeted on mobile robots impose a huge security threat. In this study, we propose the first practical robot intrusion detection system framework called RIDS, which is capable of detecting, identifying and quantifying both types of attacks. We conduct experiments on Khepera testbed which runs a motion planning mission. Our evaluation results show satisfactory detection performance under high significance levels with negligible detection delays. Future work will focus on designing and synthesizing computationally efficient intrusion response algorithms after detection.

REFERENCES

- [1] 2009. Executive Summary of World Robotics 2009. http://www.dis.uniroma1.it/~deluca/rob1_en/2009_WorldRobotics_ExecSummary.pdf. (2009).
- [2] 2016. Car Hacking Research: Remote Attack Tesla Motors. Keen Security Lab of Tencent. <http://keenlab.tencent.com/en/2016/09/19/Keen-Security-Lab-of-Tencent-Car-Hacking-Research-Remote-Attack-to-Tesla-Cars/>. (2016).
- [3] 2016. K-Team Mobile Robotics - Khepera III. <http://www.k-team.com/mobile-robotics-products/old-products/khepera-iii>. (2016).
- [4] 2016. K-Team Mobile Robotics - KoreBot II. <http://www.k-team.com/mobile-robotics-products/old-products/korebot-ii>. (2016).
- [5] Kahraman D Akdemir, Deniz Karakoyunlu, Taskin Padir, and Berk Sunar. 2010. An emerging threat: eve meets a robot. In *Trusted Systems*.
- [6] Vinay A Bavdekar, Anjali P Deshpande, and Sachin C Patwardhan. 2011. Identification of process and measurement noise covariance for state and parameter estimation using extended Kalman filter. *Journal of Process control* (2011).
- [7] Nicola Bezzo, James Weimer, Miroslav Pajic, Oleg Sokolsky, George J Pappas, and Insup Lee. 2014. Attack resilient state estimation for autonomous robotic systems. In *IROS*.
- [8] Miguel Castro, Barbara Liskov, et al. 1999. Practical Byzantine fault tolerance. In *OSDI*.

- [9] Stephen Checkoway, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, Stefan Savage, Karl Koscher, Alexei Czeskis, Franziska Roesner, Tadayoshi Kohno, et al. 2011. Comprehensive Experimental Analyses of Automotive Attack Surfaces. In *USENIX Security*.
- [10] Jie Chen, Ron J Patton, and Hong-Yue Zhang. 1996. Design of unknown input observers and robust fault detection filters. *Internat. J. Control* (1996).
- [11] Yue Cheng, Hao Ye, Yongqiang Wang, and Donghua Zhou. 2009. Unbiased minimum-variance state estimation for linear systems with unknown input. *Automatica* (2009).
- [12] Jung Jin Cho, Yong Chen, and Yu Ding. 2004. Redundancy Analysis of Linear Sensor Systems and Its Applications. <https://www.orchampion.org/content/download/55235/522615/file/redundancy.pdf>. (2004).
- [13] EYEEY Chow and A Willsky. 1984. Analytical redundancy and the design of robust failure detection systems. *IEEE Transactions on automatic control* (1984).
- [14] Jonny Carlos da Silva, Abhinav Saxena, Edward Balaban, and Kai Goebel. 2012. A knowledge-based system approach for sensor fault modeling, detection and mitigation. *Expert Systems with Applications* (2012).
- [15] Mohamed Darouach and Michel Zasadzinski. 1997. Unbiased minimum variance estimation for systems with unknown exogenous inputs. *Automatica* (1997).
- [16] Rodrigo Maximiano Antunes de Almeida, Luis Henrique de Carvalho Ferreira, and Carlos Henrique Valério. 2013. Microkernel development for embedded systems. *Journal of Software Engineering and Applications* (2013).
- [17] Giuseppe De Nicolao, Giovanni Sparacino, and Claudio Cobelli. 1997. Nonparametric input estimation in physiological systems: problems, methods, and case studies. *Automatica* (1997).
- [18] Marco Di Natale. 2008. Understanding and using the Controller Area network. (2008).
- [19] Hugh Durrant-Whyte and Tim Bailey. 2006. Simultaneous localization and mapping. *IEEE robotics & automation magazine* (2006).
- [20] Hamza Fawzi, Paulo Tabuada, and Suhas Diggavi. 2014. Secure estimation and control for cyber-physical systems under adversarial attacks. *IEEE Trans. Automat. Control* (2014).
- [21] Anita M Flynn. 1985. Redundant sensors for mobile robot navigation. (1985).
- [22] M Hou and RJ Patton. 1998. Optimal filtering for systems with unknown inputs. *IEEE Trans. Automat. Control* (1998).
- [23] Todd E Humphreys, Brent M Ledvina, Mark L Psiaki, Brady W O'Hanlon, and Paul M Kintner Jr. 2008. Assessing the spoofing threat: Development of a portable GPS civilian spoofer. In *ION GNSS*.
- [24] IDC. 2016. MANUFACTURING Press Release. <http://www.idc.com/getdoc.jsp?containerId=prUS41046916>. (2016).
- [25] Andrew H Jazwinski. 2007. *Stochastic processes and filtering theory*. Courier Corporation.
- [26] Leopoldo Jetto, Sauro Longhi, and Giuseppe Venturini. 1999. Development and experimental validation of an adaptive extended Kalman filter for the localization of mobile robots. *Robotics and Automation, IEEE Transactions on* (1999).
- [27] Thomas Kailath, Ali H Sayed, and Babak Hassibi. 2000. *Linear estimation*. Prentice Hall.
- [28] Sertac Karaman and Emilio Frazzoli. 2011. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research* (2011).
- [29] Swati Khandelwal. 2015. MalDrone - First Ever Backdoor Malware for Drones. <http://thehackernews.com/2015/01/MalDrone-backdoor-drone-malware.html>. (2015).
- [30] Hunmin Kim, Pinyao Guo, Minghui Zhu, and Peng Liu. 2017. Attack-resilient Estimation of Switched Nonlinear Cyber-Physical Systems, to appear. In *American Control Conference (ACC)*.
- [31] Peter K Kitanidis. 1987. Unbiased minimum-variance linear state estimation. *Automatica* (1987).
- [32] Karl Koscher, Alexei Czeskis, Franziska Roesner, Shwetket Patel, Tadayoshi Kohno, Stephen Checkoway, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, et al. 2010. Experimental security analysis of a modern automobile. In *S&P*.
- [33] Jayesh H Kotecha and Petar M Djuric. 2003. Gaussian particle filtering. *IEEE Transactions on signal processing* (2003).
- [34] Bhaskar Krishnamachari and Sitharama Iyengar. 2004. Distributed Bayesian algorithms for fault-tolerant event region detection in wireless sensor networks. *IEEE Trans. Comput.* (2004).
- [35] Vijay Kumar and Nathan Michael. 2012. Opportunities and challenges with autonomous micro aerial vehicles. *IJRR* (2012).
- [36] John Leonard, Jonathan How, Seth Teller, Mitch Berger, Stefan Campbell, Gaston Fiore, Luke Fletcher, Emilio Frazzoli, Albert Huang, Sertac Karaman, et al. 2008. A perception-driven autonomous urban vehicle. *Journal of Field Robotics* (2008).
- [37] Feng Li, Avinash Srinivasan, and Jie Wu. 2008. PVFS: a probabilistic voting-based filtering scheme in wireless sensor networks. *International Journal of Security and Networks* (2008).
- [38] Todd Litman. 2014. Autonomous vehicle implementation predictions. Implications for transport planning. <http://www.vtpi.org/avip.pdf>. (2014).
- [39] W Liu and I Hwang. 2011. Robust estimation and fault detection and isolation algorithms for stochastic linear hybrid systems with unknown fault input. *IET control theory & applications* (2011).
- [40] Charlie Miller and Chris Valasek. 2015. Remote exploitation of an unaltered passenger vehicle. *Black Hat USA* (2015).
- [41] Yilin Mo, Emanuele Garone, Alessandro Casavola, and Bruno Sinopoli. 2010. False data injection attacks against state estimation in wireless sensor networks. In *CDC*.
- [42] Paul Y Montgomery, Todd E Humphreys, and Brent M Ledvina. 2009. Receiver-autonomous spoofing detection: Experimental results of a multi-antenna receiver defense against a portable civil GPS spoofer. In *ITM*.
- [43] Abdelkrim Nemra and Nabil Aouf. 2009. Robust INS/GPS sensor fusion for UAV localization using SDRE nonlinear filtering. *IEEE Sensors Journal* (2009).
- [44] Miroslav Pajic, Paulo Tabuada, Insup Lee, and George J Pappas. 2015. Attack-resilient state estimation in the presence of noise. In *2015 54th IEEE Conference on Decision and Control (CDC)*.
- [45] Junkil Park, Radoslav Ivanov, James Weimer, Miroslav Pajic, and Insup Lee. 2015. Sensor attack detection in the presence of transient faults. In *ICCPS*.
- [46] Fabio Pasqualetti, Florian Dorfler, and Francesco Bullo. 2013. Attack detection and identification in cyber-physical systems. *Automatic Control, IEEE Transactions on* (2013).
- [47] Vern Paxson. 1999. Bro: a system for detecting network intruders in real-time. *Computer networks* (1999).
- [48] Jonathan Petit, Bas Stottelaar, Michael Feiri, and Frank Kargl. 2015. Remote Attacks on Automated Vehicles Sensors: Experiments on Camera and LiDAR. In *Black Hat Europe*. <https://www.blackhat.com/docs/eu-15/materials/eu-15-Petit-Self-Driving-And-Connected-Cars-Fooling-Sensors-And-Tracking-Drivers-wp1.pdf>
- [49] Jonathan Petit, B Stottelaar, M Feiri, and F Kargl. 2015. Remote attacks on automated vehicles sensors: Experiments on camera and lidar. *Black Hat Europe* (2015).
- [50] Mark L Psiaki, Brady W O'Hanlon, Jahshan A Bhatti, Daniel P Shepard, and Todd E Humphreys. 2011. Civilian GPS spoofing detection based on dual-receiver correlation of military signals. *ION GNSS* (2011).
- [51] Daniel E Rivera, Manfred Morari, and Sigurd Skogestad. 1986. Internal model control: PID controller design. *Industrial & engineering chemistry process design and development* (1986).
- [52] JA Roecker and CD McGillem. 1988. Comparison of two-sensor tracking methods based on state vector fusion and measurement fusion. *IEEE Trans. Aerospace Electron. Systems* (1988).
- [53] Martin Roesch et al. 1999. Snort: Lightweight Intrusion Detection for Networks.. In *LISA*.
- [54] Mary Ann Russon. 2016. NASA hack: AnonSec attempts to crash \$222m drone, releases secret flight videos and employee data. (2016). <http://www.ibtimes.co.uk/nasa-hack-anonsec-attempts-crash-222m-drone-releases-secret-flight-videos-employee-data-1541254>
- [55] Yasser Shoukry, Paul Martin, Yair Yona, Suhas Diggavi, and Mani Srivastava. 2015. PyCRA: Physical Challenge-Response Authentication For Active Sensors Under Spoofing Attacks. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*.
- [56] Yunmok Son, Hocheol Shin, Dongkwan Kim, Youngseok Park, Juhwan Noh, Kibum Choi, Jungwoo Choi, and Yongdae Kim. 2015. Rocking drones with intentional sound noise on gyroscopic sensors. In *USENIX Security*.
- [57] Nils Ole Tippenhauer, Christina Pöpper, Kasper Bonne Rasmussen, and Srdjan Capkun. 2011. On the requirements for successful GPS spoofing attacks. In *CCS*.
- [58] Chris Urmson, J Andrew Bagnell, Christopher R Baker, Martial Hebert, Alonzo Kelly, Raj Rajkumar, Paul E Rybski, Sebastian Scherer, Reid Simmons, Sanjiv Singh, et al. 2007. Tartan racing: A multi-modal approach to the darpa urban challenge. (2007).
- [59] Carl van Schaik and Gernot Heiser. 2007. High-performance microkernels and virtualisation on ARM and segmented architectures. In *Proceedings of the 1st International Workshop on Microkernels for Embedded Systems, Sydney, Australia*.
- [60] Christina Warrender, Stephanie Forrest, and Barak Pearlmutter. 1999. Detecting intrusions using system calls: Alternative data models. In *Security and Privacy, 1999. Proceedings of the 1999 IEEE Symposium on*.
- [61] Wikipedia. 2016. Iran-U.S. RQ-170 incident — Wikipedia, The Free Encyclopedia. (2016).
- [62] Chen Yan, X Wenyan, and Jianhao Liu. 2016. Can you trust autonomous vehicles: Contactless attacks against sensors of self-driving vehicle. *DEF CON* (2016).
- [63] Dit-Yan Yeung and Yuxin Ding. 2003. Host-based intrusion detection using dynamic and static behavioral models. *Pattern recognition* (2003).
- [64] S Yong, M Zhu, and E Frazzoli. 2015. Resilient state estimation against switching attacks on stochastic cyber-physical systems. In *CDC*.
- [65] S Z Yong, M Zhu, and E Frazzoli. 2016. A unified filter for simultaneous input and state estimation of linear discrete-time stochastic systems. *Automatica* (2016).
- [66] Zheng Yong, Minghui Zhu, and Emilio Frazzoli. 2016. Simultaneous Mode, Input and State Estimation for Switched Linear Stochastic Systems. *arXiv preprint*

A APPENDIX

A.1 Complete RIDS Design Algorithm

Algorithm 2 Complete Robot Intrusion Detection System (RIDS)

Input: Initial state estimates $\hat{\mathbf{x}}_{0|0}$; robot kinematic function $f(\cdot)$; measurement function $h(\cdot)$

Output: Detection decision; attack vector estimates

```

1: Set parameters  $w_s, w_a, c_s, c_a, \alpha_s, \alpha_a$ ;
2: Initialize;
3: for control iteration  $k \leftarrow 1$  to  $\infty$  do
4:   Receive control commands  $\mathbf{u}_{k-1}$ ;
5:   Receive sensor readings  $\mathbf{z}_k$ ;
6:   for mode  $j = 1$  to  $M$  do
7:     Run NUISE with input  $(\mathbf{u}_{k-1}, \hat{\mathbf{x}}_{k-1|k-1}, \mathbf{z}_{1,k}^j, \mathbf{z}_{2,k}^j, P_{k-1}^x)$ ,
    and generate  $(\hat{\mathbf{x}}_{k|k}^j, \hat{\mathbf{d}}_k^{s,j}, \hat{\mathbf{d}}_{k-1}^{a,j}, P_k^{x,j}, P_k^{s,j}, P_{k-1}^{a,j}, N_k^j)$ ;
8:      $\mu_k^j \leftarrow \max\{N_k^j \mu_{k-1}^j, \epsilon\}$ ;
9:   end for
10:  for mode  $j = 1$  to  $M$  do
11:     $\bar{\mu}_k^j \leftarrow \frac{\mu_k^j}{\sum_{i=1}^M \mu_k^i}$ ;
12:  end for
13:  Sensor mode accept  $J_k \leftarrow \operatorname{argmax}_j \bar{\mu}_k^j$ ;
14:  Obtain estimates and covariance matrices from  $J_k$ :  $\hat{\mathbf{x}}_{k|k} \leftarrow \hat{\mathbf{x}}_{k|k}^{J_k}, \hat{\mathbf{d}}_k^s \leftarrow \hat{\mathbf{d}}_k^{s,J_k}, \hat{\mathbf{d}}_{k-1}^a \leftarrow \hat{\mathbf{d}}_{k-1}^{a,J_k}, P_k^x \leftarrow P_k^{x,J_k}$ ;
15:   $b_k^s \leftarrow (\hat{\mathbf{d}}_k^{s,T} (P_k^{s,J_k})^{-1} \hat{\mathbf{d}}_k^s > \chi_{p=|\hat{\mathbf{d}}_k^s|}^2(\alpha_s))$ ;
16:   $b_k^a \leftarrow (\hat{\mathbf{d}}_{k-1}^{a,T} (P_{k-1}^{a,J_k})^{-1} \hat{\mathbf{d}}_{k-1}^a > \chi_{p=|\hat{\mathbf{d}}_{k-1}^a|}^2(\alpha_a))$ ;
17:  if  $b_k^s = \text{True}$  and  $\sum_{i=0}^{w_s-1} b_{k-i}^s \geq c_s$  then
18:    for each testing sensor  $t$  in mode  $J_k$  do
19:      Sensor attack vector estimate for testing sensor  $t$ :
20:       $\hat{\mathbf{d}}_{k,t}^s = \sum_{i=0}^{w_s-1} \hat{\mathbf{d}}_{k-i,t}^s / w_s$ ;
21:      if  $\hat{\mathbf{d}}_{k,t}^{s,T} (P_{k,t}^{s,J_k})^{-1} \hat{\mathbf{d}}_{k,t}^s \geq \chi_{p=|\hat{\mathbf{d}}_{k,t}^s|}^2$  then
22:        Confirm sensor attack on sensor  $t$ ;
23:      end if
24:    end for
25:  end if
26:  if  $b_k^a = \text{True}$  and  $\sum_{i=0}^{w_a-1} b_{k-i}^a \geq c_a$  then
27:    Confirm actuator attack;
28:    for each actuator  $t$  in all do
29:      Actuator attack vector estimate for actuator  $t$ :
30:       $\hat{\mathbf{d}}_{k-1,t}^a = \sum_{i=0}^{w_a-1} \hat{\mathbf{d}}_{k-1-i,t}^a / w_a$ ;
31:    end for
32:  end if
33:  return Confirmed attack type(s) and attack target(s); sensor attack vector estimates  $\hat{\mathbf{d}}_{k,t}^s$  ( $t \in \{\text{testing sensors in mode } J_k\}$ ); actuator attack vector estimates  $\hat{\mathbf{d}}_{k-1,t}^a$  ( $t \in \{1, \dots, n\}$ );
34: end for

```

A.2 NUISE Algorithm Derivation

Minimum variance unbiased state and unknown input estimation is first introduced in [31] with indirect feedthrough only. This result is extended by many research. A general parameterized gain matrix is derived in [15], and direct feedthrough unknown input estimation is integrated into the system in [11, 22]. Paper [65] analyze the stability of the system with direct and indirect feedthrough unknown input. The estimator with indirect feedthrough unknown input has been applied to system fault detection [10] without noise and [17, 39] with noise. The estimator with direct and indirect feedthrough unknown input is applied to attack detection [66] with noise in which the attack location is unknown. However, all the current research is limited to linear dynamic systems. The proposed NUISE is an extension of the above references to nonlinear systems. It is also an extension of the extended Kalman filters [25] for state estimation of nonlinear systems by integrating unknown input estimation. This is the first time to study the state and unknown input estimation on a class of stochastic nonlinear systems.

To find an optimal estimate, we first define what is the meaning of being optimal. The optimality contains two properties. Firstly, the estimate is unbiased; i.e., its expected value is equal to the targeted value. Secondly, the estimate has the minimum error covariance matrix; i.e., estimation error variance must be minimized given information.

We will derive the NUISE through 4 steps: 1) actuator attack estimation, 2) state prediction, 3) state estimation, 4) sensor attack estimation. In each intermediate step, estimation error and covariance matrix are calculated to find the optimal estimates.

Consider the following system which contains (4) as a special case:

$$\begin{aligned}
 \mathbf{x}_{k+1} &= f_k^j(\mathbf{x}_k, \mathbf{u}_k + \mathbf{d}_k^{a,j}) + \zeta_k^j \\
 \mathbf{z}_{1,k}^j &= h_{1,k}^j(\mathbf{x}_k) + \mathbf{d}_k^{s,j} + \xi_{1,k}^j \\
 \mathbf{z}_{2,k}^j &= h_{2,k}^j(\mathbf{x}_k) + \xi_{2,k}^j
 \end{aligned} \tag{5}$$

where attack $\mathbf{d}_k^{s,j}$ and $\mathbf{d}_k^{a,j}$ represent sensor attack and actuator attack. Testing sensor readings $\mathbf{z}_{1,k}^j$ might be modified by attack vector $\mathbf{d}_k^{s,j}$. Reference sensor readings $\mathbf{z}_{2,k}^j$ is assumed to be clean in mode j . We omit mode index j in the NUISE derivation because each NUISE is associated with fixed j . Dynamic system (5) can be linearized into

$$\begin{aligned}
 \mathbf{x}_{k+1} &\simeq A_k \mathbf{x}_k + B_k \mathbf{u}_k + G_k \mathbf{d}_k^a + \zeta_k \\
 \mathbf{z}_{1,k} &\simeq C_{1,k} \mathbf{x}_k + \mathbf{d}_k^s + \xi_{1,k} \\
 \mathbf{z}_{2,k} &\simeq C_{2,k} \mathbf{x}_k + \xi_{2,k}
 \end{aligned} \tag{6}$$

where

$$\begin{aligned}
 A_k &\triangleq \frac{\partial f_k}{\partial \mathbf{x}} \Big|_{\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k + \hat{\mathbf{d}}_{k-1}^a}, B_k \triangleq \frac{\partial f_k}{\partial \mathbf{u}} \Big|_{\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k + \hat{\mathbf{d}}_{k-1}^a}, \\
 C_{1,k} &\triangleq \frac{\partial h_{1,k}}{\partial \mathbf{x}} \Big|_{\hat{\mathbf{x}}_{k|k-1}}, C_{2,k} \triangleq \frac{\partial h_{2,k}}{\partial \mathbf{x}} \Big|_{\hat{\mathbf{x}}_{k|k-1}} \\
 G_k &\triangleq \frac{\partial f_k}{\partial \mathbf{d}^a} \Big|_{\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k + \hat{\mathbf{d}}_{k-1}^a}.
 \end{aligned}$$

Attack $\mathbf{d}_{k-1}^a$ estimation: Given unbiased previous state estimate $\hat{\mathbf{x}}_{k-1|k-1}$, we can predict the current state using the known

kinematic function $f_k(\cdot)$ as follows

$$\hat{\mathbf{x}}_{k|k-1}^* = f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}).$$

The estimate error is described by

$$\tilde{\mathbf{x}}_{k|k-1}^* = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k-1}^* = A_{k-1}\tilde{\mathbf{x}}_{k-1|k-1} + G_{k-1}\mathbf{d}_{k-1}^a + \zeta_{k-1}.$$

Noticeably, the estimation is biased, i.e., $\mathbb{E}[\hat{\mathbf{x}}_{k|k-1}^*] \neq \mathbf{x}_{k|k-1}$ because we do not consider possible unknown attacks yet $G_{k-1}\mathbf{d}_{k-1}^a \neq 0$. To have the unbiased state prediction, it is now needed to find the estimate of actuator attack. The expected output without considering the actuator attack will be $C_{2,k}\hat{\mathbf{x}}_{k|k-1}^*$. The informational discrepancy between what we expected and what we actually obtain $\mathbf{z}_{2,k} - C_{2,k}\hat{\mathbf{x}}_{k|k-1}^*$ shows us the effect of attack $\mathbf{d}_{k-1}^a$ and thus this term is used to estimate it. Actuator attacks are estimated linearly from sensor output bias

$$\begin{aligned} \hat{\mathbf{d}}_{k-1}^a &= M_{2,k}(\mathbf{z}_{2,k} - C_{2,k}f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})) \\ &= M_{2,k}(C_{2,k}A_{k-1}\tilde{\mathbf{x}}_{k-1|k-1} + C_{2,k}G_{k-1}\mathbf{d}_{k-1}^a \\ &\quad + C_{2,k}\zeta_{k-1} + \xi_{2,k}) \end{aligned}$$

where the estimator gain $M_{2,k}$ represents a weight average of sensor bias based on the trustfulness of each sensor. The unknown input estimate is unbiased, i.e., $\mathbb{E}[\hat{\mathbf{d}}_{k-1}^a] = \mathbf{d}_{k-1}^a$ if $\mathbb{E}[\tilde{\mathbf{x}}_{k-1|k-1}] = 0$, and $M_{2,k}C_{2,k}G_{k-1} = I$. In order to achieve optimal estimates, matrix gain M_k should be carefully chosen with minimum variance. To do this, consider the sensor output bias

$$\tilde{\mathbf{z}}_{2,k} = C_{2,k}(G_{k-1}\mathbf{d}_{k-1}^a + A_{k-1}\tilde{\mathbf{x}}_{k-1|k-1} + \zeta_k) + \xi_k$$

where $\mathbb{E}[C_{2,k}A_{k-1}\tilde{\mathbf{x}}_{k-1|k-1} + C_{2,k}\zeta_{k-1} + \xi_k] = 0$ and its covariance is calculated by

$$\tilde{R}_{2,k}^* \triangleq \mathbb{E}[\tilde{\mathbf{z}}_{2,k}\tilde{\mathbf{z}}_{2,k}^T] = C_{2,k}\tilde{P}_{k-1}C_{2,k}^T + R_{2,k}$$

where $\tilde{P}_k \triangleq A_{k-1}P_{k-1}^x A_{k-1}^T + Q_{k-1}$. We choose the matrix M_k using the Gauss Markov theorem [27] as

$$M_{2,k} = (G_{k-1}^T C_{2,k}^T \tilde{R}_{2,k}^{*-1} C_{2,k} G_{k-1})^{-1} G_{k-1}^T C_{2,k}^T \tilde{R}_{2,k}^{*-1}$$

which satisfies $M_{2,k}C_{2,k}G_{k-1} = I$. We assume that $G_{k-1}^T C_{2,k}^T \tilde{R}_{2,k}^{*-1} C_{2,k} G_{k-1}$ is invertible. Attack vector estimation error covariance is $P_{k-1}^a \triangleq \mathbb{E}[\tilde{\mathbf{d}}_{k-1}^a(\tilde{\mathbf{d}}_{k-1}^a)^T] = M_{2,k}\tilde{R}_{2,k}^* M_{2,k}^T$.

State prediction: Estimate $\hat{\mathbf{x}}_{k|k-1}^*$ was calculated under a partial knowledge on the attack. Since now we have the estimate $\mathbf{d}_{k-1}^a$ of attack, we can update the state estimate

$$\hat{\mathbf{x}}_{k|k-1} = f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1} + \hat{\mathbf{d}}_{k-1}^a)$$

and it is now unbiased $\mathbb{E}[\hat{\mathbf{x}}_{k|k-1}] = \mathbf{x}_k$ since $\mathbb{E}[\hat{\mathbf{d}}_{k-1}^a] = \mathbf{d}_{k-1}^a$. For the next step, we find the state prediction error covariance matrix

$$P_{k|k-1}^x = \bar{A}_{k-1}P_{k-1}^x \bar{A}_{k-1}^T + \bar{Q}_{k-1} \quad (7)$$

where $\bar{A}_{k-1} = (I - G_{k-1}M_{2,k}C_{2,k})A_{k-1}$ and $\bar{Q}_{k-1}^j = (I - G_{k-1}M_{2,k}C_{2,k})Q_{k-1}^j(I - G_{k-1}M_{2,k}C_{2,k})^T + G_{k-1}M_{2,k}R_{2,k}M_{2,k}^T G_{k-1}^T$.

State estimation: Predicted state $\hat{\mathbf{x}}_{k|k-1}$ is not perfect because of process and measurement errors. To have the estimate more accurate, we correct the state estimate using sensor readings. Here,

we again utilize the difference between the newly predicted output $C_{2,k}\hat{\mathbf{x}}_{k|k-1}$ and real sensor output $\mathbf{z}_{2,k}$ to reflect the effect of unknown noises:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + L_k(\mathbf{z}_{2,k} - h_{2,k}(\hat{\mathbf{x}}_{k|k-1}))$$

where the state estimate is unbiased $\mathbb{E}[\hat{\mathbf{x}}_{k|k}] = \mathbf{x}_k$ and the estimate gain matrix L_k will be chosen such that the new estimate $\hat{\mathbf{x}}_{k|k}$ has a smaller error variance. Error dynamic and covariance are

$$\tilde{\mathbf{x}}_{k|k} = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k} = (I - L_k C_{2,k})\tilde{\mathbf{x}}_{k|k-1} - L_k \xi_{2,k}$$

and

$$\begin{aligned} P_k^x &= (I - L_k C_{2,k})P_{k|k-1}^x(I - L_k C_{2,k})^T + L_k R_{2,k} L_k^T \\ &\quad - (I - L_k C_{2,k})G_{k-1}M_{2,k}R_{2,k}L_k^T \\ &\quad - L_k R_{2,k}M_{2,k}^T G_{k-1}^T(I - L_k C_{2,k})^T. \end{aligned}$$

To achieve the optimal estimates, we solve the variance minimization program: $\min_{L_k} \text{tr}(P_k^x)$. We can take derivative the objective function with respect to the decision variable L_k and set it to zero to find the solution: $L_k = (C_{2,k}P_{k|k-1}^x + R_{2,k}M_{2,k}^T G_{2,k-1}^T)^T \tilde{R}_{2,k}^{-1}$ where $\tilde{R}_{2,k} \triangleq C_{2,k}P_{k|k-1}^x C_{2,k}^T + R_{2,k} + C_{2,k}G_{k-1}M_{2,k}R_{2,k} + R_{2,k}M_{2,k}^T G_{k-1}^T C_{2,k}^T$ must be invertible.

Attack $\mathbf{d}_k^{s,j}$ estimation: Given $\hat{\mathbf{x}}_{k|k}$, the linear estimation for unknown input $\mathbf{d}_k^s$ can be

$$\begin{aligned} \hat{\mathbf{d}}_k^s &= M_{1,k}(\mathbf{z}_{1,k} - h_{1,k}(\hat{\mathbf{x}}_{k|k})) \\ &= M_{1,k}(C_{1,k}\tilde{\mathbf{x}}_{k|k} + \mathbf{d}_k^s + \xi_{1,k}) \end{aligned} \quad (8)$$

where the estimate is unbiased $\mathbb{E}[\hat{\mathbf{d}}_k^s] = \mathbf{d}_k^s$ if $M_{1,k} = I$. This also can be found by Gauss Markov theorem. By the theorem, the optimal estimate is

$$M_{1,k} \triangleq (\tilde{R}_{1,k}^{-1})^{-1} \tilde{R}_{1,k}^{-1} = I$$

where $\tilde{R}_{1,k} = C_{1,k}P_{k|k-1}^x C_{1,k}^T + R_{1,k}$. Covariance matrices are found by

$$P_k^s = \tilde{R}_{1,k}$$

Likelihood of the mode: It is natural that the predicted output must be matched with the measured output if the mode j is the true mode. For $\forall j$, we quantify the discrepancy between the predicted output and the measured output as follows

$$v_k^j = \mathbf{z}_{2,k} - h_{2,k}^j(\hat{\mathbf{x}}_{k|k-1}^j).$$

We approximate the output error v_k^j as a multivariate Gaussian random variable. Then, the likelihood function is given by

$$\begin{aligned} \mathcal{N}_k^j &\triangleq \mathcal{P}(y_k|j = \text{true}) = \mathcal{N}(v_k^j; 0, \bar{P}_{k|k-1}^j) \\ &= \frac{\exp(-(v_k^j)^T (\bar{P}_{k|k-1}^j)^{\dagger} v_k^j / 2)}{(2\pi)^{n^j/2} |\bar{P}_{k|k-1}^j|_+^{1/2}} \end{aligned}$$

where $\bar{P}_{k|k-1}^j = C_{2,k}^j P_{k|k-1}^{x,j} (C_{2,k}^j)^T + R_{2,k}^j - C_{2,k}^j G_{k-1}^j M_{2,k}^j R_{2,k}^j - R_{2,k}^j (M_{2,k}^j)^T (G_{k-1}^j)^T (C_{2,k}^j)^T$ is the error covariance matrix of v_k^j and $n^j = \text{Rank}(\bar{P}_{k|k-1}^j)$. Notations $\dagger$ and $|\cdot|_+$ refer pseudoinverse and pseudodeterminant, respectively. By the Bayes' theorem, the a posteriori probability is $\mu_k^j \triangleq \mathcal{P}(j = \text{true}|y_k, \dots, y_0) =$

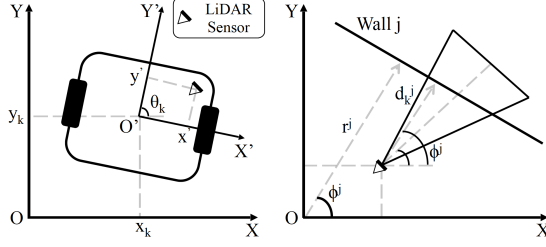


Figure 8: LiDAR sensor measurement model.

$\frac{\mathcal{P}(y_k | j=\text{true})\mathcal{P}(j=\text{true} | y_{k-1}, \dots, y_0)}{\sum_{i=1}^M \mathcal{P}(y_k | j=\text{true})\mathcal{P}(j=\text{true} | y_{k-1}, \dots, y_0)} = \frac{\mathcal{N}_k^j \mu_{k-1}^j}{\sum_{i=1}^M \mathcal{N}_k^i \mu_{k-1}^i}$. However, such update might allow that some μ_k^j converge to zero. To prevent this, we modify the posterior probability update to

$$\bar{\mu}_k^j = \frac{\mu_k^j}{\sum_{i=1}^M \mu_k^i},$$

where $\mu_k^j = \max\{\mathcal{N}_k^j \mu_{k-1}^j, \epsilon\}$ and $\epsilon > 0$ is a pre-selected small constant preventing the vanishment of the mode probability. The last step is to generate the state, attack vector, and mode estimates of the maximum a posteriori mode.

Algorithm 3 shows the complete nonlinear unknown input and state estimation algorithm.

A.3 Kinematic Model and Measurement Models

Kinematic model The kinematic model of Khepera includes three states: (x, y) is the robot location at a 2-D plane and θ is its heading. The control commands are determined by two variables: v_L and v_R are the speeds of the left and right wheels, respectively. Considered with actuator attack $\mathbf{d}_{k-1}^a = [d_{k-1}^{a,L}, d_{k-1}^{a,R}]^T$ on the left and right wheel, the kinematic model can be presented as:

$$\begin{aligned} x_k &= x_{k-1} + T \cos \theta_{k-1} (v_L + d_{k-1}^{a,L} + v_R + d_{k-1}^{a,R})/2 + \zeta_{k-1}^x \\ y_k &= y_{k-1} + T \sin \theta_{k-1} (v_L + d_{k-1}^{a,L} + v_R + d_{k-1}^{a,R})/2 + \zeta_{k-1}^y \\ \theta_k &= \theta_{k-1} + T(v_R + d_{k-1}^{a,R} - v_L - d_{k-1}^{a,L})/D + \zeta_{k-1}^\theta \end{aligned} \quad (9)$$

where $\zeta_{k-1} = [\zeta_{k-1}^x, \zeta_{k-1}^y, \zeta_{k-1}^\theta]^T$ is assumed to be zero mean Gaussian process noises, and D is the distance between the left and right wheel on Khepera.

Measurement model The sensor readings include data from three sensors: $\mathbf{z}_k = [\mathbf{z}_{k,I}, \mathbf{z}_{k,W}, \mathbf{z}_{k,L}]^T$ where $\mathbf{z}_{k,I}$ is from IPS, $\mathbf{z}_{k,W}$ is from wheel encoder, and $\mathbf{z}_{k,L}$ is from LiDAR.

IPS sensor directly measures the states of Khepera, hence, the measurement model can be directly specified by:

$$\mathbf{z}_{k,I} = \mathbf{x}_k + \mathbf{d}_{k,I}^s + \xi_{k,I} \quad (10)$$

where $\xi_{k,I} = [\xi_{k,I}^x, \xi_{k,I}^y, \xi_{k,I}^\theta]^T$ refers to measurement noises from IPS and $\mathbf{d}_{k,I}^s = [d_{k,I}^{s,x}, d_{k,I}^{s,y}, d_{k,I}^{s,\theta}]^T$ refers to the sensor attack vector on IPS.

⁴Notations $\dagger$ and $|\cdot|_+$ refer pseudoinverse and pseudodeterminant, respectively. n^j refers to the rank of $\bar{\mathbf{P}}_{k|k-1}^j$.

Algorithm 3 Nonlinear Unknown Input and State Estimation Algorithm (NUISE)

Input: $\mathbf{u}_{k-1}, \hat{\mathbf{x}}_{k-1|k-1}, \mathbf{z}_{1,k}^j, \mathbf{z}_{2,k}^j, \mathbf{P}_{k-1}^x$

Output: $\hat{\mathbf{x}}_{k|k}^j, \hat{\mathbf{d}}_{k-1}^{s,j}, \hat{\mathbf{d}}_{k-1}^{a,j}, \mathbf{P}_{k-1}^{x,j}, \mathbf{P}_{k-1}^{s,j}, \mathbf{P}_{k-1}^{a,j}, \mathcal{N}_k^j$

- 1: Initialize;
- 2: $\tilde{\mathbf{P}}_{k-1}^j \leftarrow \mathbf{A}_{k-1}^j \mathbf{P}_{k-1}^x (\mathbf{A}_{k-1}^j)^T + \mathbf{Q}_{k-1}^j$;
- 3: $\tilde{\mathbf{R}}_{2,k}^{*,j} \leftarrow \mathbf{C}_{2,k}^j \tilde{\mathbf{P}}_{k-1}^j (\mathbf{C}_{2,k}^j)^T + \mathbf{R}_{2,k}^j$;
- 4: $\mathbf{M}_{2,k}^j \leftarrow ((\mathbf{G}_{k-1}^j)^T (\mathbf{C}_{2,k}^j)^T (\tilde{\mathbf{R}}_{2,k}^{*,j})^{-1} \mathbf{C}_{2,k}^j \mathbf{G}_{k-1}^j)^{-1} (\mathbf{G}_{k-1}^j)^T (\mathbf{C}_{2,k}^j)^T (\tilde{\mathbf{R}}_{2,k}^{*,j})^{-1}$;
- 5: $\hat{\mathbf{d}}_{k-1}^{a,j} \leftarrow \mathbf{M}_{2,k}^j (\mathbf{z}_{2,k}^j - \mathbf{C}_{2,k}^j f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}))$;
- 6: $\mathbf{P}_{k-1}^{a,j} \leftarrow \mathbf{M}_{2,k}^j \tilde{\mathbf{R}}_{2,k}^{*,j} (\mathbf{M}_{2,k}^j)^T$;
- 7: $\hat{\mathbf{x}}_{k|k-1}^j \leftarrow f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1} + \hat{\mathbf{d}}_{k-1}^{a,j})$;
- 8: $\tilde{\mathbf{A}}_{k-1}^j \leftarrow (\mathbf{I} - \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{C}_{2,k}^j) \mathbf{A}_{k-1}^j$;
- 9: $\tilde{\mathbf{Q}}_{k-1}^j \leftarrow (\mathbf{I} - \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{C}_{2,k}^j) \mathbf{Q}_{k-1}^j (\mathbf{I} - \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{C}_{2,k}^j)^T + \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{R}_{2,k}^j (\mathbf{M}_{2,k}^j)^T (\mathbf{G}_{k-1}^j)^T$;
- 10: $\mathbf{P}_{k|k-1}^{x,j} \leftarrow \tilde{\mathbf{A}}_{k-1}^j \mathbf{P}_{k-1}^x (\tilde{\mathbf{A}}_{k-1}^j)^T + \tilde{\mathbf{Q}}_{k-1}^j$;
- 11: $\tilde{\mathbf{R}}_{2,k}^j \leftarrow \mathbf{C}_{2,k}^j \mathbf{P}_{k|k-1}^{x,j} (\mathbf{C}_{2,k}^j)^T + \mathbf{R}_{2,k}^j + \mathbf{C}_{2,k}^j \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{R}_{2,k}^j + \mathbf{R}_{2,k}^j (\mathbf{M}_{2,k}^j)^T (\mathbf{G}_{k-1}^j)^T (\mathbf{C}_{2,k}^j)^T$;
- 12: $\mathbf{L}_k^j \leftarrow (\mathbf{C}_{2,k}^j \mathbf{P}_{k|k-1}^{x,j} + \mathbf{R}_{2,k}^j (\mathbf{M}_{2,k}^j)^T (\mathbf{G}_{k-1}^j)^T)^T (\tilde{\mathbf{R}}_{2,k}^j)^{-1}$;
- 13: $\hat{\mathbf{x}}_{k|k}^j \leftarrow \hat{\mathbf{x}}_{k|k-1}^j + \mathbf{L}_k^j (\mathbf{z}_{2,k}^j - \mathbf{h}_2^j(\hat{\mathbf{x}}_{k|k-1}^j))$;
- 14: $\mathbf{P}_k^{x,j} \leftarrow (\mathbf{I} - \mathbf{L}_k^j \mathbf{C}_{2,k}^j) \mathbf{P}_{k|k-1}^{x,j} (\mathbf{I} - \mathbf{L}_k^j \mathbf{C}_{2,k}^j)^T + \mathbf{L}_k^j \mathbf{R}_{2,k}^j (\mathbf{L}_k^j)^T - (\mathbf{I} - \mathbf{L}_k^j \mathbf{C}_{2,k}^j) \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{R}_{2,k}^j (\mathbf{L}_k^j)^T - \mathbf{L}_k^j \mathbf{R}_{2,k}^j (\mathbf{M}_{2,k}^j)^T (\mathbf{G}_{k-1}^j)^T (\mathbf{I} - \mathbf{L}_k^j \mathbf{C}_{2,k}^j)^T$;
- 15: $\hat{\mathbf{d}}_k^{s,j} \leftarrow \mathbf{z}_{1,k}^j - \mathbf{h}_1^j(\hat{\mathbf{x}}_{k|k}^j)$;
- 16: $\mathbf{P}_k^{s,j} \leftarrow \mathbf{C}_{1,k}^j \mathbf{P}_k^{x,j} (\mathbf{C}_{1,k}^j)^T + \mathbf{R}_{1,k}^j$;
- 17: $\mathbf{v}_k^j \leftarrow \mathbf{z}_{2,k}^j - \mathbf{h}_2^j(\hat{\mathbf{x}}_{k|k}^j)$;
- 18: $\tilde{\mathbf{P}}_{k|k-1}^j \leftarrow \mathbf{C}_{2,k}^j \mathbf{P}_{k|k-1}^{x,j} (\mathbf{C}_{2,k}^j)^T + \mathbf{R}_{2,k}^j - \mathbf{C}_{2,k}^j \mathbf{G}_{k-1}^j \mathbf{M}_{2,k}^j \mathbf{R}_{2,k}^j - \mathbf{R}_{2,k}^j (\mathbf{M}_{2,k}^j)^T (\mathbf{G}_{k-1}^j)^T (\mathbf{C}_{2,k}^j)^T$;
- 19: $n^j \leftarrow \text{rank}(\tilde{\mathbf{P}}_{k|k-1}^j)$;
- 20: $\mathcal{N}_k^j \leftarrow \frac{1}{(2\pi)^{n^j/2} |\tilde{\mathbf{P}}_{k|k-1}^j|_+^{1/2}} \exp(-\frac{(\mathbf{v}_k^j)^T (\tilde{\mathbf{P}}_{k|k-1}^j)^{\dagger} \mathbf{v}_k^j}{2})$;

The raw data measured by the wheel encoder are the distances traveled by each wheel (l_L, l_R). For convenience reason, we convert them into robot states using previous states $\mathbf{x}_{k-1}$ before we feed the data to planner:

$$\begin{aligned} x_k &= x_{k-1} + (l_L + l_R) \cos \theta_k / 2 \\ y_k &= y_{k-1} + (l_L + l_R) \sin \theta_k / 2 \\ \theta_k &= \theta_{k-1} + (l_R - l_L) / r \end{aligned}$$

Analogously with IPS, the measurement model for the wheel encoder is specified as:

$$\mathbf{z}_{k,W} = \mathbf{x}_k + \mathbf{d}_{k,W}^s + \xi_{k,W} \quad (11)$$

where $\xi_{k,W} = [\xi_{k,W}^x, \xi_{k,W}^y, \xi_{k,W}^\theta]^T$ refers to measurement noises from the wheel encoder and $\mathbf{d}_{k,W}^s = [d_{k,W}^{s,x}, d_{k,W}^{s,y}, d_{k,W}^{s,\theta}]^T$ refers to the sensor attack vector on the wheel encoder.

The LiDAR sensor is placed on top of the robot with a shift distance of $[x', y']^T$ from the origin O' as shown in the left of Figure 8. Raw sensor readings returned from LiDAR are the distances between LiDAR and surrounding walls and obstacles (see Figure 7(b)). Given the LiDAR readings, we process the raw data into the perpendicular distance l_k^j from each boundary wall $j \in \{1, 2, 3, 4\}$ and the orientation $\theta_{k,L}$ of Khepera. Specifically, we recognize the straight line segments using raw distances from all direction, and calculate the distances to each wall as follows:

$$l_k^j = r^j - (x_k + x' \sin \theta_k + y' \cos \theta_k) \cos \phi^j - (y_k - x' \cos \theta_k + y' \sin \theta_k) \sin \phi^j + d_{k,L}^{s,j} + \xi_{k,L}^j \quad (12)$$

where $\xi_{k,L} = [\xi_{k,L}^j]^T, j \in \{1, 2, 3, 4\}$ refers to measurement noises from LiDAR. Distance r^j and angle ϕ^j for each wall is known in advance as the map information. Using ϕ^j of each wall and the 240 degrees of range, we can also infer the angle of the robot. We use the distances to each wall and the angle as the readings from LiDAR: $\mathbf{z}_{k,L} = [l_k^j, \theta_k]^T, j \in \{1, 2, 3, 4\}$. In outdoor environments, LiDAR measurement model can be obtained using more complicated simultaneous localization and mapping (SLAM) algorithms [19]. For demonstration purpose, we apply a simple transformation in the indoor environment [26].

A.4 Separating Actuator Attack Vector

Without loss of generality, we consider a robot with two actuators such as Khepera. During actuator attack vector estimation, we obtain $\hat{\mathbf{d}}_k^a = [\hat{d}_k^L, \hat{d}_k^R]^T$, with error covariance P_k^a . In Algorithm 1, we test

$$(\hat{\mathbf{d}}_k^a)^T (P_k^a)^{-1} \hat{\mathbf{d}}_k^a \geq \chi_{p=2}(\alpha) \quad (13)$$

to determine the existence of actuator attacks. Threshold $\chi_{p=2}(\alpha)$ is a Chi-square test value with degree of freedom $p = 2$ and confidence level α .

In order to confirm actuator attack on each actuator, we need to separately conduct Chi-square test $\hat{d}_k^L$, and $\hat{d}_k^R$, with corresponding marginal variances $P_k^a(1, 1)$, and $P_k^a(2, 2)$:

$$\begin{aligned} (\hat{d}_k^L)^T (P_k^a(1, 1))^{-1} \hat{d}_k^L &\geq \chi_{p=1}^2(\alpha) \\ (\hat{d}_k^R)^T (P_k^a(2, 2))^{-1} \hat{d}_k^R &\geq \chi_{p=1}^2(\alpha). \end{aligned} \quad (14)$$

However, a positive testing result in (13) does not guarantee a positive testing result in (14) because off-diagonal terms of matrix P_k^a are neglected in (14). The explanation is shown as follows:

$$\begin{aligned} (\hat{\mathbf{d}}_k^a)^T (P_k^a)^{-1} \hat{\mathbf{d}}_k^a &= (\hat{d}_k^L)^T (P_k^a)^{-1} (1, 1) \hat{d}_k^L \\ &\quad + (\hat{d}_k^L)^T (P_k^a)^{-1} (1, 2) \hat{d}_k^R + (\hat{d}_k^R)^T (P_k^a)^{-1} (2, 1) \hat{d}_k^L \\ &\quad + (\hat{d}_k^R)^T (P_k^a)^{-1} (2, 2) \hat{d}_k^R \\ (\hat{d}_k^L)^T (P_k^a(1, 1))^{-1} \hat{d}_k^L &= (\hat{d}_k^L)^T (P_k^a(1, 1))^{-1} \hat{d}_k^L \\ (\hat{d}_k^R)^T (P_k^a(2, 2))^{-1} \hat{d}_k^R &= (\hat{d}_k^R)^T (P_k^a(1, 1))^{-1} \hat{d}_k^R \end{aligned} \quad (15)$$

Note that $(\hat{\mathbf{d}}_k^a)^T (P_k^a)^{-1} \hat{\mathbf{d}}_k^a = (\hat{d}_k^L)^T (P_k^a(1, 1))^{-1} \hat{d}_k^L + (\hat{d}_k^R)^T (P_k^a(2, 2))^{-1} \hat{d}_k^R$ if P_k^a is a diagonal matrix.

Another problem for the separation is that Chi-square test threshold is nonlinear. For instance, $\chi_{p=1}^2(0.01) = 6.635$ and $\chi_{p=2}^2(0.01) = 9.210$. Suppose P_k^a is a diagonal matrix and the test scores after separation are $(\hat{d}_k^L)^T (P_k^a(1, 1))^{-1} \hat{d}_k^L = 5$ and $(\hat{d}_k^R)^T (P_k^a(2, 2))^{-1} \hat{d}_k^R = 5$. The actuator attack would be detected by (13) but not by (14).

Hence, we conduct the Chi-square test for the aggregated actuator attacks.

A.5 Building RIDS on UAV

To further demonstrate the general applicability of RIDS, we will elaborate how to build an RIDS on UAV. Consider an UAV which is mounted with an inertial navigation system (IMU) and a GPS. The state of the UAV can be specified as $\mathbf{x}_k \triangleq [x_k, y_k, z_k, v_k^x, v_k^y, v_k^z, \phi_k, \theta_k, \psi_k]^T$, which denotes displacements, velocities, and angles on X, Y and Z axes. Inputs $\mathbf{u}_k \triangleq [p, q, r, a^x, a^y, a^z]^T$ are rotation rates and accelerations on three axes. The kinematic model of the UAV is given in (4) where the kinematic function $f(\cdot)$ is as follows:

$$f(\mathbf{x}_k, \mathbf{u}_k) = \begin{bmatrix} \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} + T R_k^T \begin{bmatrix} v_k^x \\ v_k^y \\ v_k^z \end{bmatrix} \\ \begin{bmatrix} v_k^x \\ v_k^y \\ v_k^z \end{bmatrix} + T \begin{bmatrix} a_k^x + v_k^y r - v_k^z q + g s(\theta_k) \\ a_k^y - v_k^x r + v_k^z p - g c(\theta_k) s(\phi_k) \\ a_k^z + v_k^x q - v_k^y p - g c(\theta_k) c(\phi_k) \end{bmatrix} \\ \begin{bmatrix} \phi_k \\ \theta_k \\ \psi_k \end{bmatrix} + T \begin{bmatrix} 1 & s(\phi_k) \tan(\theta_k) & c(\phi_k) t(\theta_k) \\ 0 & c(\phi_k) & -s(\phi_k) \\ 0 & s(\phi_k) \sec(\theta_k) & c(\phi_k) \sec(\theta_k) \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \end{bmatrix}$$

where R_k is

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\phi_k) & s(\phi_k) \\ 0 & -s(\phi_k) & c(\phi_k) \end{bmatrix} \begin{bmatrix} c(\theta_k) & 0 & -s(\theta_k) \\ 0 & 1 & 0 \\ s(\theta_k) & 0 & c(\theta_k) \end{bmatrix} \begin{bmatrix} c(\psi_k) & s(\psi_k) & 0 \\ -s(\psi_k) & c(\psi_k) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

and $c(\cdot)$, and $s(\cdot)$ refer to $\cos(\cdot)$ and $\sin(\cdot)$ respectively. GPS only measures the location of UAV, hence the measurement function is $h(\mathbf{x}_k) = [x_k, y_k, z_k]^T$. An IMU generates full states, hence the measurement function can be described as $h(\mathbf{x}_k) = \mathbf{x}_k$. With the above model, we can apply RIDS to detect the attacks on UAV.

A.6 Detection Results under Attack Scenarios

More detection results from RIDS under attack scenarios in Table 3 are shown in Figure ???. Figure ??? shows the detection output when there is neither actuator attack nor sensor attack. Estimation results in plot 1-4 show nearly zero attack vector estimates. The Chi-square test statistics shown in plot 5 and 7 indicate both actuator and sensor attack remain under the threshold, except some occasional spikes. After the sliding window filtering, plot 6 and 8 indicates an attack silence.